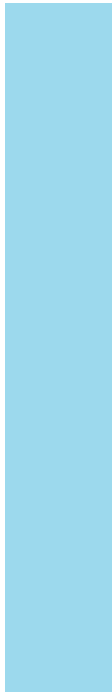


Introduction to Machine Learning.





Contents

1	Introduction	3
2	Supervised learning	4
2.1	Basics	4
2.1.1	Classification and Regression	4
2.1.2	Train, Test and Validation datasets	5
2.1.3	Data preprocessing	8
2.2	Performance metrics for classification	9
2.3	Algorithms	12
2.3.1	Decision Trees	12
2.3.2	Random Forests	18
2.3.3	k -Nearest Neighbor classifier	19
2.3.4	Logistic Regression	20
3	Unsupervised learning	23
3.1	Clustering	23
3.1.1	k -Means	23
3.1.2	Agglomerative clustering	27
3.1.3	DBSCAN	31
3.1.4	Performance measures	33
3.2	Dimensionality Reduction	35
3.2.1	Principal Component Analysis (PCA)	35
3.2.2	t-Distributed Stochastic Neighbor Embedding (t-SNE)	37
3.2.3	Uniform Manifold Approximation and Projection (UMAP)	38
4	Deep learning	39
4.1	Network Training	39
4.1.1	Training	39
4.2	Neural Networks	41
4.2.1	Multilayer Perceptron	41
4.2.2	Convolutional Neural Networks (CNN)	43
4.2.3	Activation functions	49
4.2.4	Other layers	52



1 Introduction

In the last years, the quantity of accessible data has increased exponentially thanks to the availability of more efficient and affordable data storage devices together with the less expensive and more powerful processing capacities. The field of physics is not alien to the data growth, e.g. astronomy¹.

However, data alone is useless. In order to acquire knowledge, it is imperative to analyze and process the data to distill the information. Nonetheless, the analysis of data in the current amount scale requires sophisticated methods. **Machine learning is a field which aims to develop such methods.** In concrete, **machine learning designs algorithms that "learn" from data to make predictions or decisions.** The concept of learning is a bit vague at this stage, but usually what is meant is the following: given some data (e.g. images of dogs and chickens), the machine learning algorithm processes the data so that it detects patterns (e.g. does it have hair? does it have a beak?) that are useful on the final task (classify the image as a dog or a chicken). **Finally, we would test if the algorithm is able to make the right decision on unseen data.** **The ability to recognize the right patterns is what it is usually understood by learning, since it has not been explicitly programmed to detect such patterns, but it has learned to identify them.**

Usually, machine learning is divided in the following three categories (see Figure 1):

- **Supervised learning:** The objective is to learn from labeled data. In this setting, there is access to the desired response (label) and this is used to aid the model to produce the correct output.
- **Unsupervised learning:** The aim is to explore the structure of data and distill information out of it. In contrast to supervised learning, we do not have access to any kind of label. Thus, the exploration of the data is done without any kind of guidance.
- **Reinforcement learning:** The goal is to develop a system (agent) that interacts with and adapts to the surrounding environment. Similarly to supervised learning, the agent has access to a reward signal that indicates how well it is doing. However, unlike supervised learning, this reward signal is not a label or a desired target response; instead, it serves as a measure of the current performance. For instance, in the context of a chess-playing agent, the reward might be based on the outcome of the game (win or loss), rather than the specific movements it should have taken at each turn.

During this tutorial we will only touch on supervised and unsupervised learning. The tutorial is organized as follows:

- Section 2 will focus on supervised learning algorithms and how to assess the quality of their predictions.
- Section 3 will discuss unsupervised learning, more concretely dimensionality reduction and clustering techniques.
- Section 4 will touch upon deep learning. We will describe the basics of deep learning and we will study the multi-layer perceptron (MLP) and convolutional neural network (CNN) architectures.

¹<https://datascience.codata.org/articles/10.5334/dsj-2015-011/>

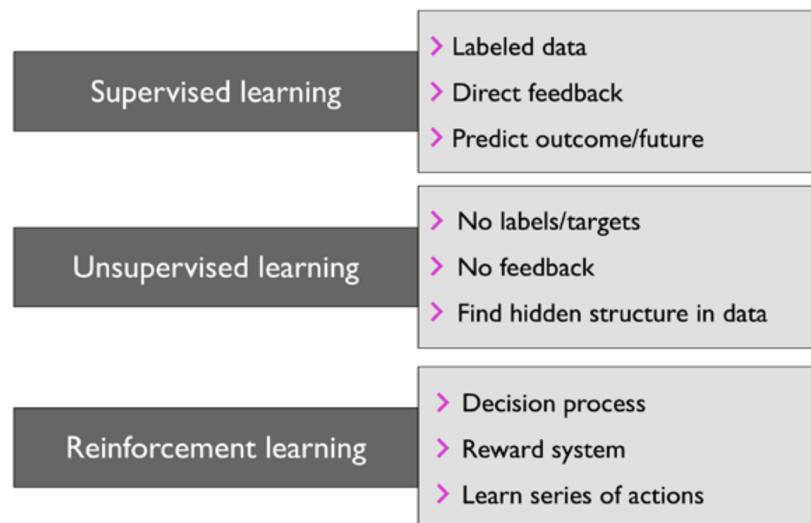


Figure 1: The three types of machine learning. *Figure from Raschka et al. [2022].*

2 Supervised learning

As stated in the introduction, supervised learning is a subfield of machine learning, in which the dataset includes target outputs, aiding the learning process. Based on this data, the model learns to generalize and accurately respond to various inputs.

Next, we will introduce the basic concepts. Subsequently, we will delve into some of the most well-known algorithms used in supervised learning. Specifically, our focus will be on those used for **classification**, although some can also be applied to **regression tasks**. Additionally, we will discuss various metrics used to evaluate the quality of predictions made by the model.

2.1 Basics

2.1.1 Classification and Regression

The goal of supervised learning is to learn a model using **labeled** data, $\mathcal{X} = \{x_i, y_i\}_{i=1}^N$, that it is able to predict outputs correctly and generalize to unseen data. The key difference with unsupervised learning is the access to the labels, $\{y_i\}_{i=1}^N$, which are the target responses that our model aims to predict. Depending on the nature of the labels we will distinguish between two tasks:

- **Classification:** In this case, the set of possible labels is discrete. The model assigns to each observation, x_i , a class or category from a (generally finite) set $\mathcal{L}$ (see Figure 2a). When $\mathcal{L}$ only contains two categories, then we often use the term binary classification; otherwise, we talk about multi-class classification. A typical example of binary classification is a spam detector that determines whether an e-mail is spam or not.
- **Regression:** In this scenario, the space of the target variables is continuous (see Figure 2b). For instance, predicting the weight of a person or the value of a house are regression tasks.

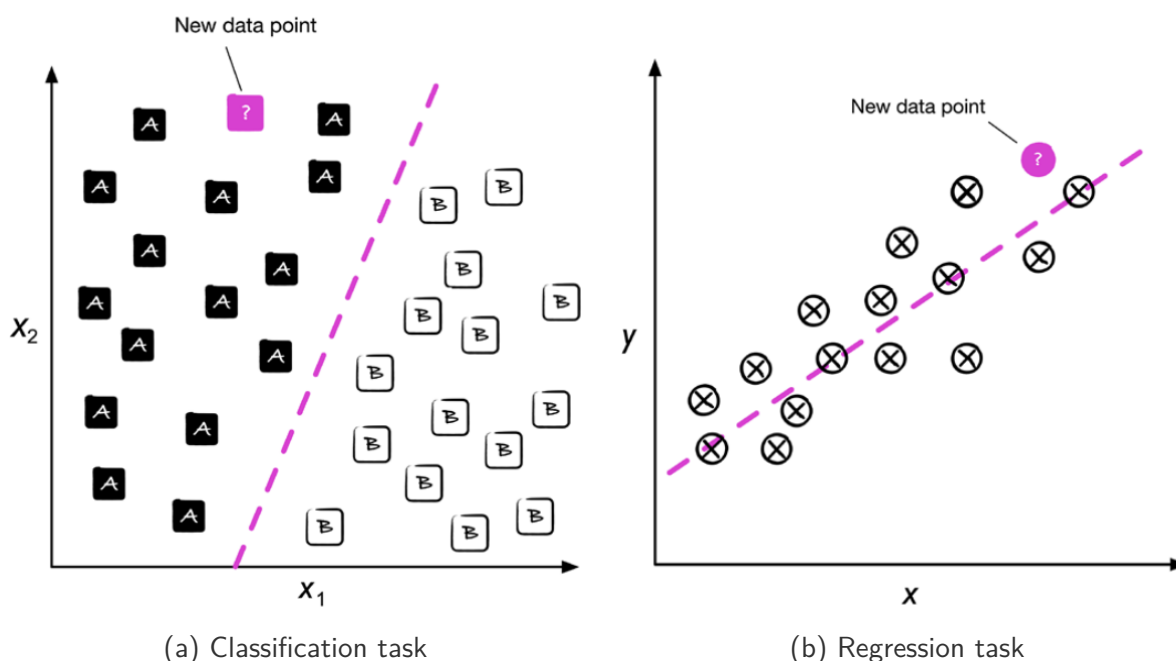


Figure 2: **2a** Classification task, whose goal is to distinguish the letters “A” and “B” (discrete output space). **2b** Regression task, whose goal is to find the line that best describes the data (continuous output space). Figures from [Marsland \[2014\]](#)

Note that the difference between these tasks only lies in the continuity or discreteness of the label space (output space) and not in the data (input space) itself. Thus, **both tasks may use discrete and continuous features, or even a combination of both types, as input.** In this tutorial, our focus will be on the classification task. However, some of the methods that we will study can also be applied to regression tasks.

2.1.2 Train, Test and Validation datasets

The purpose of learning is to improve the predictive power of a model. One common approach to evaluate the effectiveness of learning is by comparing the model’s predictions with the actual labels, known as the ground truth. So it is for supervised learning tasks. Most of the machine learning algorithms undergo a training phase, where the model is trained on a dataset, often referred as *training set*. The goal is for the model to be able to predict the correct output for this dataset.

However, we also want our model to perform well on data it has not seen before, meaning we want our model to generalize. Since we can not measure how well it generalizes with the training set, we use a second dataset called *test set*. We use this test set to compare the predicted outputs with the targets to assess the performance of the model. Note that the test set is not used to update the predictive power of the model. If we were to do so, we would not be able to accurately measure the generalization ability of the model. Thus, it is imperative that **the test set is never used in the training phase.**

In some cases, a model may perform specially well on the training data but poorly on the test set. This situation is known as *overfitting*. In overfitting, the model learns spurious or

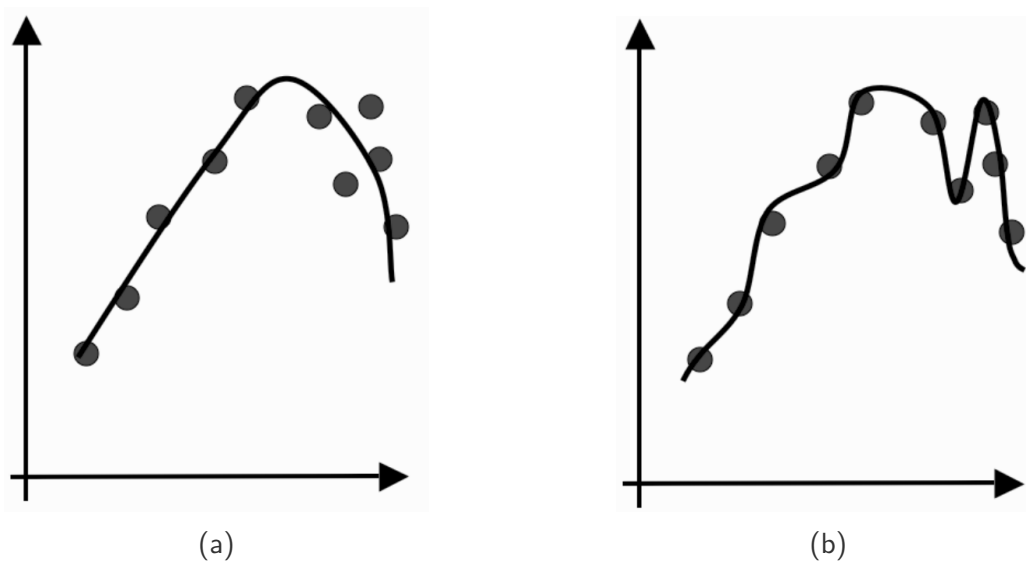


Figure 3: The effect of overfitting is that rather than finding the generating function (as shown on 3a), the model matches the inputs perfectly, including the noise in them (on 3b). This reduces the generalisation capabilities of the model. *Figure from Marsland [2014].*

noisy patterns specific of the training set, which do not generalize to unseen data (see Figure 3).

On the other hand, **underfitting** occurs when **the model fails to learn the patterns of the data that are necessary to solve the task**. This may be due to **the simplicity of the model** or **an insufficient duration of the training phase**.

Overfitting can not be detected using the training set alone. Neither can it be detected with the test set, since this must not be involved in the training phase. To detect overfitting, a third dataset, called **validation dataset**, is used to **evaluate the generalization performance at each timestep**.

Hence, given a dataset, it is common practice to split it in train, test and validation sets (as shown in Figure 4). The validation set is not only advantageous to detect overfitting, but it can also be used to select the parameters that optimize the generalization capability of the model. Note, that the validation set is used to validate the learning process, and for this very same reason, the test set can not be used for this purpose.

Cross Validation Cross validation is a technique used to assess the performance and generalization ability of a model. It involves partitioning the dataset into multiple subsets, typically called folds. The model is trained on a subset of the data and validated on the remaining portions iteratively. This process allows for a more robust estimation of the model's performance compared to a single train-test split. Common methods of cross validation include *k-fold cross validation*, where the data is divided into k subsets and the model is trained and validated k times, each time using a different fold as the validation set. This technique helps in obtaining a more reliable estimate of the model's performance and can be particularly useful when the dataset size is limited or when there is concern about the randomness of the data split. Cross validation aids in detecting overfitting and provides insights into how well the model generalizes to unseen data.

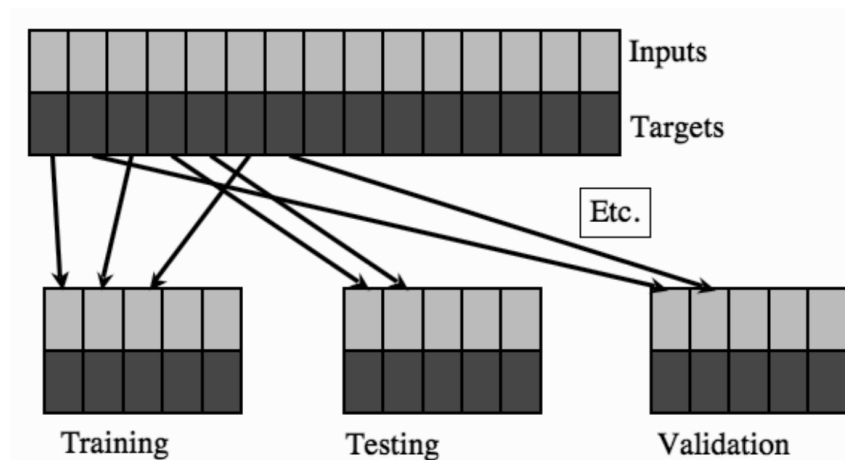


Figure 4: The dataset is split into different sets, some for training, some for validation, and some for testing. *Figure from [Marsland \[2014\]](#).*

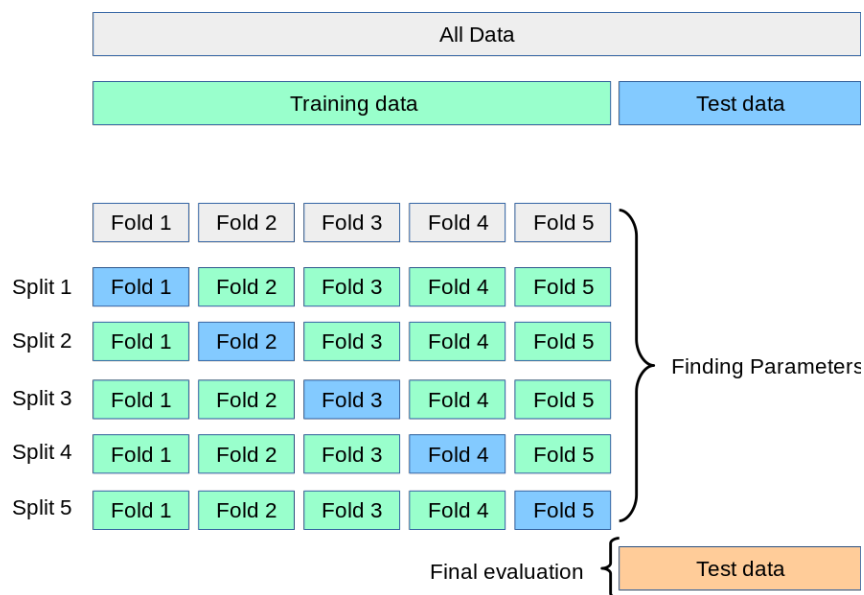


Figure 5: Representation of k -fold cross validation with $k = 5$. The training set is divided into 5 different folds, from which the model is trained on 4 folds while leaving one out for validation. This process is repeated 5 times, with each fold serving as the validation set exactly once. Cross-validation serves to identify overfitting tendencies and assesses the evaluation ability of the model. *Figure from [sklearn](#).*

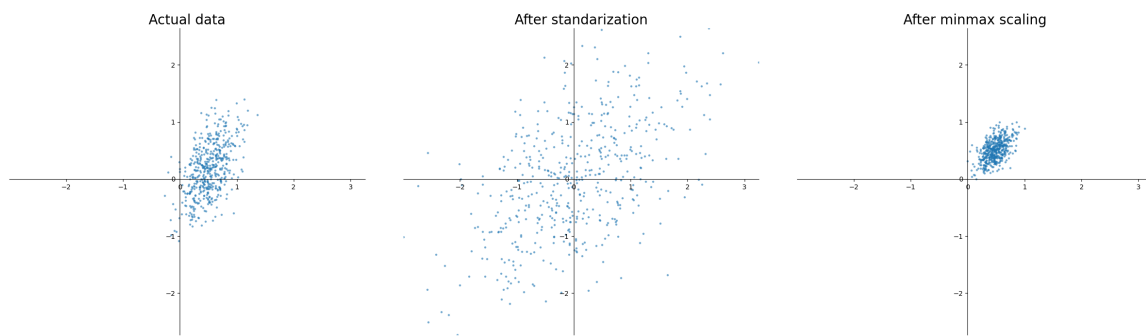


Figure 6: Comparison between data, minmax normalization and data standardization.

2.1.3 Data preprocessing

Finally, we will touch upon data preprocessing. Generally raw data does not have the appropriate shape or form for the optimal performance of learning algorithms. Thus, **it is essential to apply some transformation to the data to accommodate it to a learning algorithm**. This process is known as **preprocessing**.

Some algorithms require that the features of the data are in the same scale for optimal performance (e.g. k-nearest neighbor classifier). Two ways of normalizing the data to have equal scale are (see Figure 6):

- *Minmax normalization*: maps all the features onto the range $[0,1]$ by applying

$$x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)}.$$

- *Standardization*: It centers the data (zero becomes the mean) with a unit standard deviation. In contrast to minmax scaling, the transformed data is not mapped to any specific range. Formally, standardization is achieved by applying

$$x_{standard} = \frac{x - \text{mean}(x)}{\text{std}(x)}.$$

Remark 2.1 Usually, the normalization/standardization is applied to each feature individually; however, this is not always the case. For instance, when we are dealing with image data, the usual way of preprocessing is by scaling/standardizing the pixel intensity values across all channels, often using a range of $[0, 1]$ or $[-1, 1]$ for the normalization of the data. Additionally, it is common practice to normalize each channel separately, typically by subtracting the mean and dividing by the standard deviation of the pixel intensities within each channel. This approach helps retain the relative intensity information and improves the performance of learning algorithms on image data.

Nonetheless, in other scenarios, it may be better to normalize all channels together, i.e., by computing the mean and standard deviation across all pixels of all images. This is useful when we want to enforce a more consistent global scaling across the dataset, especially in cases where channel-specific variations are not meaningful or when we want to reduce computational

overhead in real-time applications. This approach can also be beneficial when working with grayscale images or when treating the image data as a single distribution rather than distinct channels.

Other preprocessing techniques may be focused on removing noise or discarding redundant features (e.g dimensionality reduction, Section 3.2). In other cases, we may encounter missing features for certain data samples. The estimation of these features is also part of the preprocessing. Note that the preprocessing step is not exclusive of supervised learning, but applies to all types of learning.

2.2 Performance metrics for classification

Given a set of classes $\mathcal{L} = \{\ell_1, \dots, \ell_L\}$, classification is the problem of identifying which of the classes an observation belongs to. Thus, the classification problem aims to find the function $f : \mathcal{X} \rightarrow \mathcal{L}$ that satisfies $f(x) = y$ for each object $x \in \mathcal{X}$ with label $y \in \mathcal{L}$. Let $\hat{f} : \mathcal{X} \rightarrow \mathcal{L}$ be a classifier which aims to approximate the ideal classification function f . An important question to ask is: how can we evaluate how good $\hat{f}$ approximates f ? In this section, we will analyze different performance metrics for classification.

All metrics are evaluated on a dataset $\mathcal{X} = \{(x_1, y_1), \dots, (x_n, y_n)\}$, where x_i and y_i represent the features and the ground truth label/class of sample i . Let us denote by $\hat{y}_i$ the prediction made by our classifier, i.e. $\hat{f}(x_i) = \hat{y}_i$. In the following we will present different evaluation metrics.

Accuracy: The accuracy is equal to the fraction of correctly labeled samples.

$$\text{Acc}(y, \hat{y}) = \frac{|\{i : y_i = \hat{y}_i\}|}{n}$$

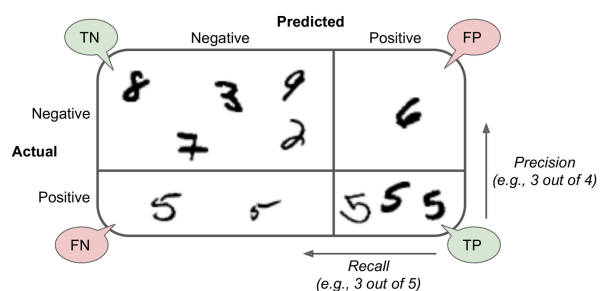
Accuracy is not a suitable measure when dealing with unbalanced datasets, where some classes are much more frequent than others. For example, consider a dataset of 100 patients, of which only 10 have cancer. A dummy classifier that always predicts a negative outcome (i.e., no cancer) would achieve a high accuracy of 90%. However, this classifier fails to fulfill the desired task of detecting patients with cancer.

Confusion matrix: The confusion matrix, CM , is a $L \times L$ matrix where the rows index the true classes and the columns correspond to the predicted classes. Each entry counts how often a sample labeled with the class indexed by the row has been classified as the class indexed by the column.

$$CM_{\ell_i \ell_j} = |\{(x_k, y_k) \in \mathcal{X} : y_k = \ell_i, \hat{f} = \ell_j\}|$$

With the confusion matrix we may detect which classes tend to be mislabeled and also identify with which classes they are likely to be confused with. For instance, Figure 7b depicts the confusion matrix of a model trained on CIFAR10, a dataset composed of images of 10 different categories. Analyzing the confusion matrix, we observe that the class 'truck' tends to be confused more often with the class 'automobile' than with any other class.

Figure 7a illustrates a confusion of a binary classifier, i.e. a classifier which only distinguishes between two classes, $\mathcal{L} = \{-1, +1\}$. When the classifier is binary we differentiate between four kinds of predictions:



(a) Confusion matrix binary classifier. Figure from (b) Confusion matrix multiclass classifier. Figure from [GÄlron \[2019\]](#) [researchgate.net](#)

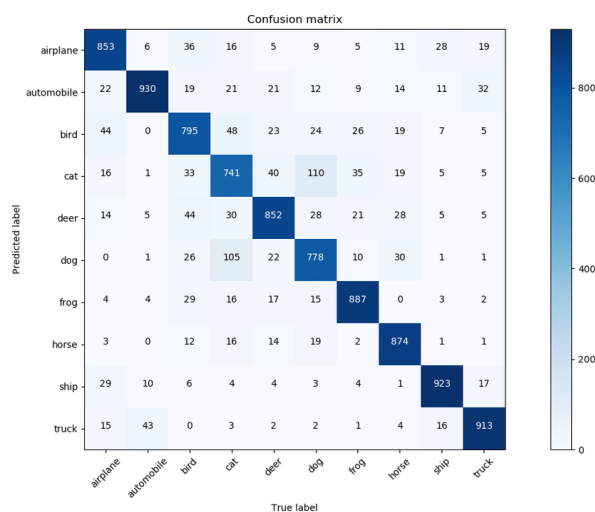


Figure 7: **7a** Confusion matrix illustration of a binary classifier that distinguishes if a digit is a 5 (positive sample) or not (negative sample). Elements in the diagonal represent the correctly labeled samples, while the rest of entries represent the falsely predictions, i.e. False Negatives (FN) and False Positives (FP). From the entries of the confusion matrix, the recall and precision metrics can be computed. **7b** Confusion matrix of model trained on CIFAR10, a dataset of images with 10 classes.

- True positives (TP): Samples of the data set where $y = \hat{y} = 1$.
- True negatives (TN): Samples of the data set where $y = \hat{y} = -1$.
- False positives (TP): Samples of the data set where $-1 = y \neq \hat{y} = 1$.
- False negatives (TP): Samples of the data set where $1 = y = \hat{y} = -1$.

For the binary case, the confusion matrix has the following shape:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}.$$

Recall: This measure is only applied in binary classification. The measure determines how many of the positive samples are captured by the positive predictions.

$$\text{recall} = \frac{TP}{TP + FN}$$

Let's revisit the scenario of the dummy cancer classifier that identifies every patient as a healthy person, with the class 'has cancer' considered as the positive one. In this case, the recall will be equal to 0 since $TP=0$ (every patient is classified as healthy), indicating the lowest possible score. This appropriately reflects the model's failure to detect any cases of cancer.

Consider now a different classifier with the following confusion matrix

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix} = \begin{bmatrix} 70 & 20 \\ 1 & 9 \end{bmatrix}. \quad (1)$$

In this case the recall score is equal to

$$\frac{TP}{TP + FN} = \frac{9}{9 + 1} = 0.9.$$

This score is relatively high considering the fact that we detect many cancers for patients who actually do not have it. The recall measure ignores the false positives, which may not be always ideal.

Precision Similar to recall, precision is a metric applied in binary classification tasks. It calculates the proportion of samples predicted as positive that are actually positive:

$$\text{precision} = \frac{TP}{TP + FP}$$

Using the values of the confusion matrix in (1), the precision equal to

$$\frac{TP}{TP + FP} = \frac{9}{9 + 20} = \frac{9}{29} \approx 0.31.$$

Though the recall had a rather high value, the precision is relatively low. This illustrates how precision and recall complement each other, as precision focuses on the proportion of true positives among all positive predictions, while recall focuses on capturing all positive instances, minimizing false negatives.

Therefore, recall may be a more appropriate measure if the priority is to minimize false negatives (e.g., in medical diagnostics), while precision may be more suitable when minimizing false positives is essential

F-score: The F-score summarizes the recall and precision metrics by computing their harmonic mean:

$$F = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

In contrast to the regular mean, the harmonic mean emphasizes much more the low values. Therefore, the F-score will be high only if both precision and recall are high. The F-score can be generalized to multiclass classification in 3 different ways:

- Macro averaging: computes the unweighted per-class F-scores. This gives equal weight to all classes, no matter what their size is.
- Weighted averaging: computes the mean of the per-class F-scores, weighted by their normalized class size.
- Micro averaging: computes the total number of false positives, false negatives, and true positives over all classes. Then calculates precision, recall, and F-score using the formulas for binary classification.

ROC AUC: The receiver operating characteristic (ROC) curve is another measure for binary classifiers. Usually, binary classifier decision functions are of the form

$$\hat{f}(x) = \begin{cases} +1 & \text{if } \tilde{f}(x) > \text{threshold} \\ -1 & \text{otherwise} \end{cases}. \quad (2)$$

The function $\tilde{f}$ is the score function of the classifier. The ROC curve plots the false positive rate (ratio of negative instances that are incorrectly classified as positive) against the true positive rate (another name for recall) for different threshold values.

$$\text{False positive rate} = \frac{FP}{FP + TN}, \quad (\text{Recall}) \text{ True positive rate} = \frac{TP}{TP + FN}$$

A performance measure of the classifier is the area under the curve (AUC). Intuitively, the area under the curve increases, the easier is to separate the classes by a hyperplane after being mapped by $\tilde{f}$. A perfect classifier will have a ROC AUC equal to 1, whereas a purely random classifier will have a ROC AUC equal to 0.5. Figures 8 and 9 show the ROC curve for a poor and a perfect classifier.

2.3 Algorithms

In this section we walk over the most popular and common machine learning algorithms for classification, though some of them can be also applied for regression. The material of this section was based on [Raschka et al. \[2022\]](#); [MÄijller and Guido \[2016\]](#).

2.3.1 Decision Trees

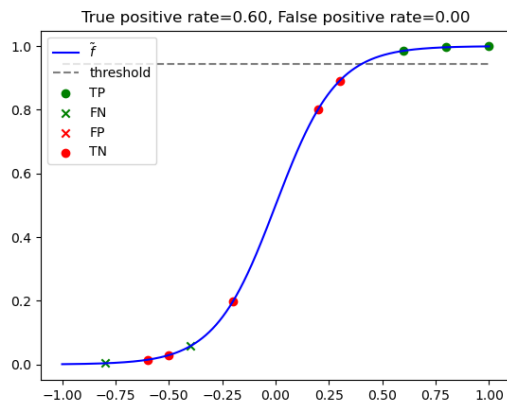
Decision trees are tree-like models that split the data iteratively according to some parameters based on the features of the data (see Figure 10). Each node in the tree represents a decision. The leaves of the tree represent the final output.

Each decision node of the tree performs the most informative data-split about the target variable. In concrete, each decision node maximizes the information gain (IG) which is defined as

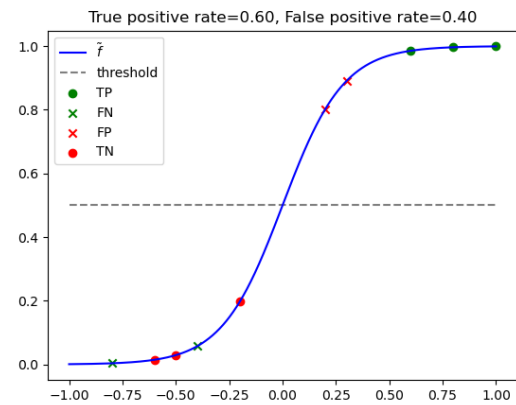
$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right}),$$

where D_p is the parent node (current decision node), D_{left} and D_{right} represent its left and right children (we have assumed a binary tree, but each decision node could have a higher degree). N_{left} , N_{right} and N_p are the total number of training examples at the left, right and parent nodes, and $I(\cdot)$ is the impurity measure of the node. Intuitively, the impurity measure indicates the diversity of classes that end up in a certain node. The higher the diversity, the higher the impurity. Thus, the information gain is calculated as the difference between the impurity of the parent node and the sum of the impurities of the children nodes. A larger information gain indicates that the split reduces the diversity of classes in the children nodes, effectively guiding the decision tree to find the optimal split.

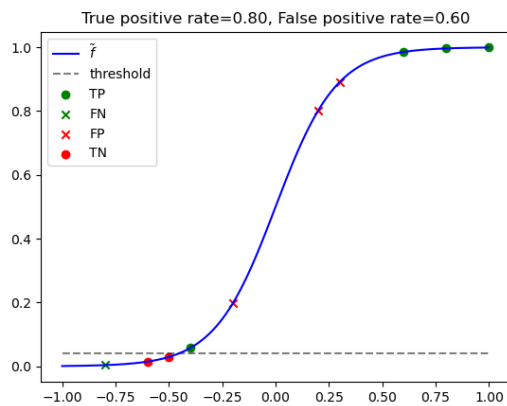
Two common impurity measures or splitting criteria are the Gini impurity (I_G) and the entropy (I_H). Let $p(\ell|t)$ be the proportion of the examples that belong to label $\ell \in \mathcal{L} = \{1, \dots, L\}$ for a particular node, t .



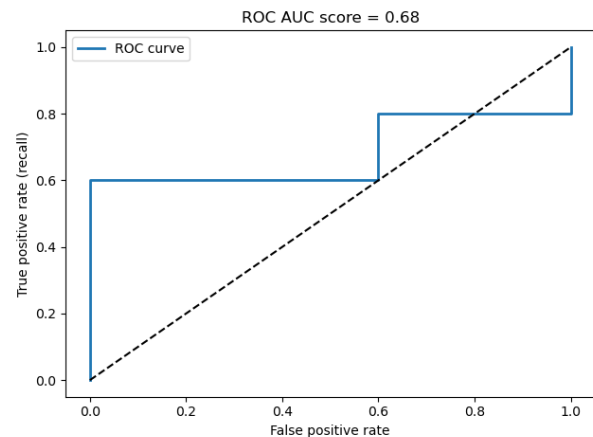
(a)



(b)

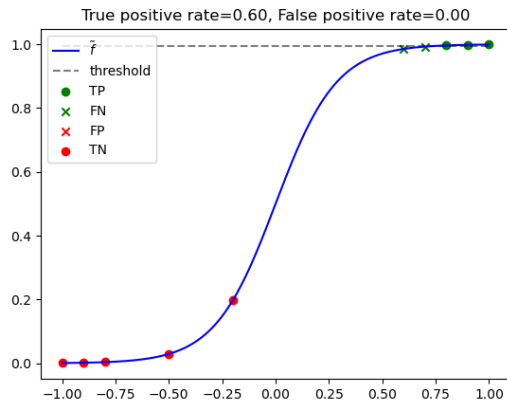


(c)

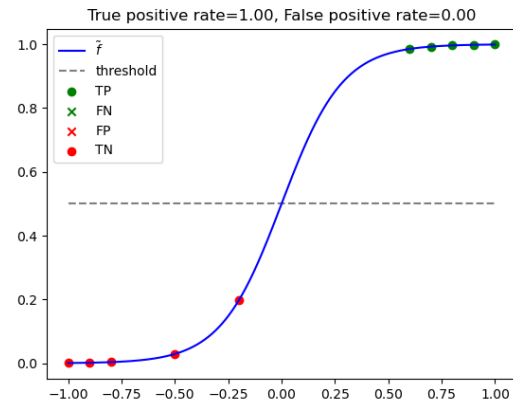


(d)

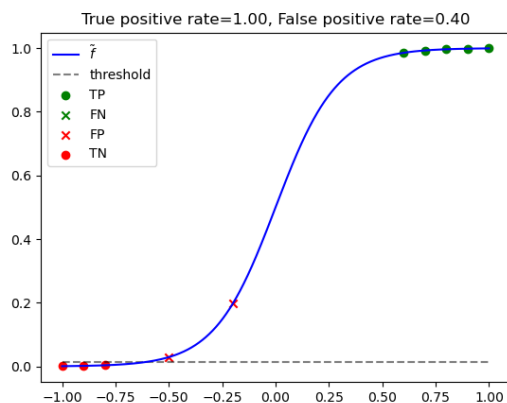
Figure 8: 8a, 8b, 8c depict the score function $\tilde{f}$ with different threshold values to determine the prediction. The green points represent the image of the positive samples via $\tilde{f}$. The image of the negative ones is shown in red. The crosses “x” indicate that the sample has been misclassified. 8d ROC curve, which plots the false positive rate against the true positive rate for all possible thresholds. The area under the curve is equal to 0.68, which indicates a poor performance.



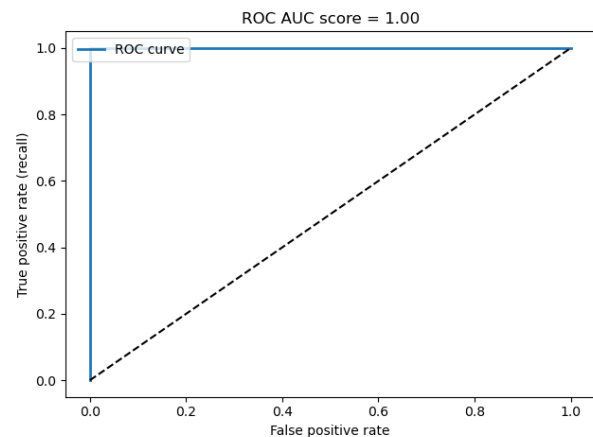
(a)



(b)



(c)



(d)

Figure 9: As in Figure 8, Figures 9a, 9b, 9c depict the score function $\tilde{f}$ with different threshold values to determine the prediction. In this case, the classifier is perfect, since the image of the classes is linearly separable after applying $\tilde{f}$. Indeed, the ROC curve in 9d has an area under the curve equal to 1.

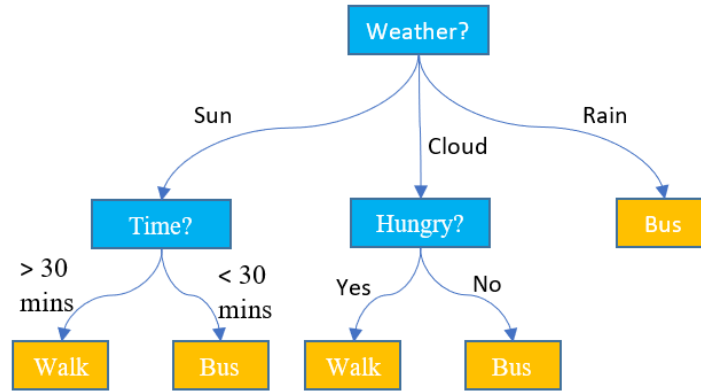


Figure 10: Decision tree illustration. The decision tree predicts the transportation method (public transport or by foot) based on different features (weather, time, hunger state). At each level, the tree branches depending on the value of one of the features. [Source](#)

- The entropy criterion attempts to minimize the entropy at each child node in the tree.

$$I_H(t) = - \sum_{\ell=1}^C p(\ell|t) \log(p(\ell|t))$$

The entropy attains its maximum if the distribution modeled by $p(\cdot|t)$ is uniform, while it attains each minimum if the distribution $p(\cdot|t)$ is equal to 0 for all the classes except for one class $\hat{\ell}$, i.e. it is minimized if

$$p(\ell|t) \begin{cases} 1 & \text{if } \ell = \hat{\ell} \\ 0 & \text{otherwise} \end{cases}.$$

Thus, the minimum is attained when the node only contains elements of a single class $\hat{\ell}$. Figure 11a plots the entropy values when $\mathcal{L}$ only contains two classes. In this case, $p(\ell|t)$, is fully described by $p(\ell = 0|t)$ since $p(0|t) + p(1|t) = 1$. Indeed, the maximum is attained when $p(0|t) = p(1|t) = 0.5$ (maximum impurity) and the minimum is attained when $p(\ell = 0|t) = 1$ or $p(\ell = 1|t) = 1$ (minimum impurity).

- The Gini impurity can be understood as a criterion to minimize the probability of misclassification:

$$I_G(t) = \sum_{\ell=1}^L p(\ell|t)(1 - p(\ell|t)) = 1 - \sum_{\ell=1}^L p(\ell|t)^2.$$

As it happened for the entropy measure, it attains its minimum when a node only contains elements of a single class, and it is maximized when the classes are perfectly mixed (uniform distribution). Figure 11b plots the Gini values when $\mathcal{L}$ only contains two classes.

We can observe in Figure 11, that both impurity measures are similar in shape. In general, the performance of the tree does not get affected when these impurity measures are exchanged.

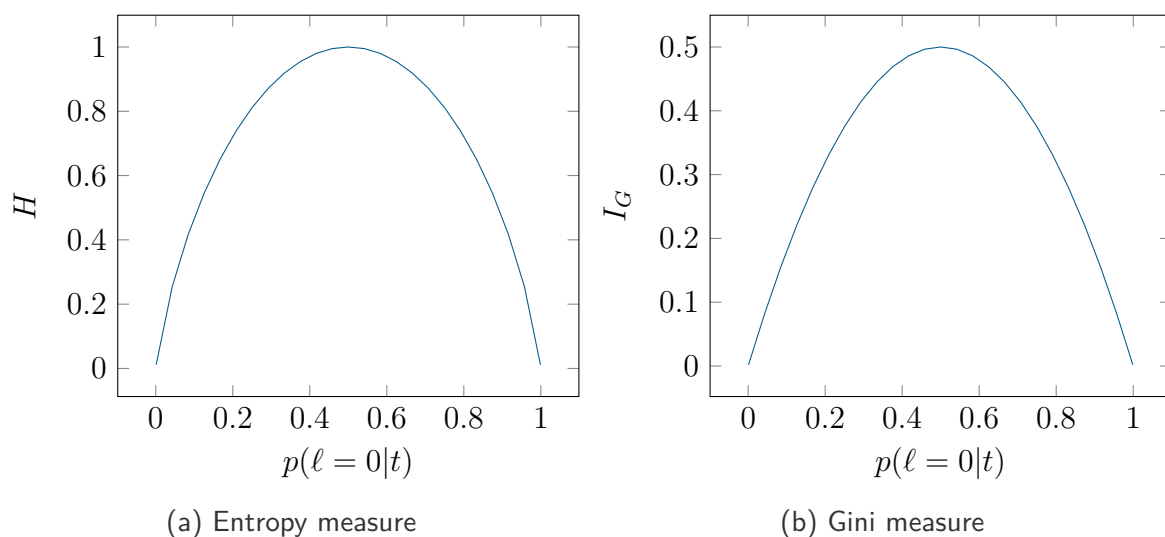


Figure 11: Entropy and Gini impurity measure values with respect to the class-membership when 2 classes are considered.

Though there are different algorithms to train a decision tree, we will focus on **the CART algorithm (Classification and Regression Trees)**.² The CART algorithm **determines the structure and splitting criteria in a recursive greedy manner** (see Algorithm 1). The algorithm starts with a single root node, corresponding to the whole input training set, and then the tree grows by adding nodes one at a time. At each iteration, a candidate node is considered and all input variables and all possible split points of this node are evaluated. The best split, i.e., the one that maximizes the information gain, is selected, generating two new nodes. This process is recursively iterated until some stopping criterion is satisfied. The most common stopping procedure is to use a minimum count on the number of training samples assigned to each leaf node. Other approaches limit the number of leaves or the depth of the tree.

For continuous features, binary splits are performed at each iteration. These splits are determined by a threshold value that partitions the feature space into two regions. This procedure divides the input space into rectangles that are further partitioned independently of each other at each iteration (see Figure 12). For categorical features, splits are defined by comparing the feature value to each of the possible values for that feature, rather than to a numeric threshold. Alternatively, one could consider a multi-way split as done in the first node of the tree. However, this may cause too little data to fall into each subtree, leading to overfitting. Therefore, it is preferable to use binary splits at each node of the tree.

Overall, the partition of the input space defined by the decision trees can be interpreted by a human thanks to the structured recursive decision rules. The human interpretability of the decision trees is considered an strength, although interpreting large decision trees can still be challenging in practice. On the other hand, decision trees tend to overfit, meaning they can capture noise in the training data and produce different decision rules with small changes in the dataset. To mitigate overfitting, stopping criteria such as restricting the depth of the

²For regression the impurity measure used is the squared difference between the targets and the predictions, i.e. $\sum_i (y_i - \hat{y})^2$. The final prediction at the leaves is defined by the mean response of all the training samples that reach the corresponding leaf

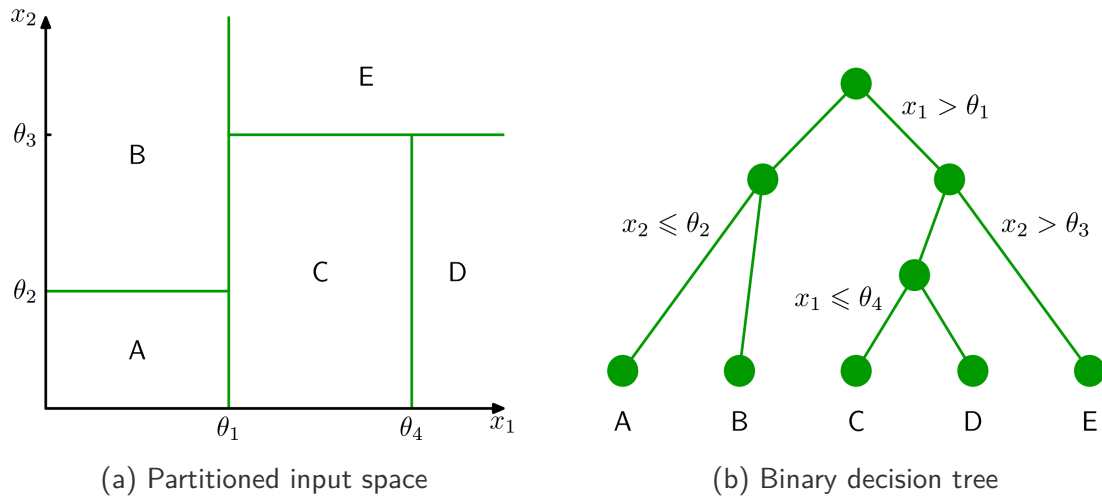


Figure 12: 12a Illustration of a two dimensional input space that has been partitioned according to the binary tree in 12b using axis-aligned boundaries. Figure from Bishop [2006].

Algorithm 1: Decision Tree Training Algorithm

Input: $\mathcal{X} = \{x_i, y_i\}_{i=1}^N$ // Data
Output: $\mathcal{T}$ // Decision Tree

- 1 $\mathcal{T} = \emptyset$ // root
- 2 **repeat**
- 3 **for** $f \in \mathbb{F}$ **do**
- 4 $IG(f|t)$ // Compute split for f with maximum IG at node t
- 5 $\hat{f} = \arg \max_{f \in \mathbb{F}} IG(f|t)$ // Select feature which maximizes IG
- 6 $\text{Split}(\mathcal{T}, \hat{f}, t)$ // Perform the split associated to $\hat{f}$ at node t
- 7 **until** stopping criterion

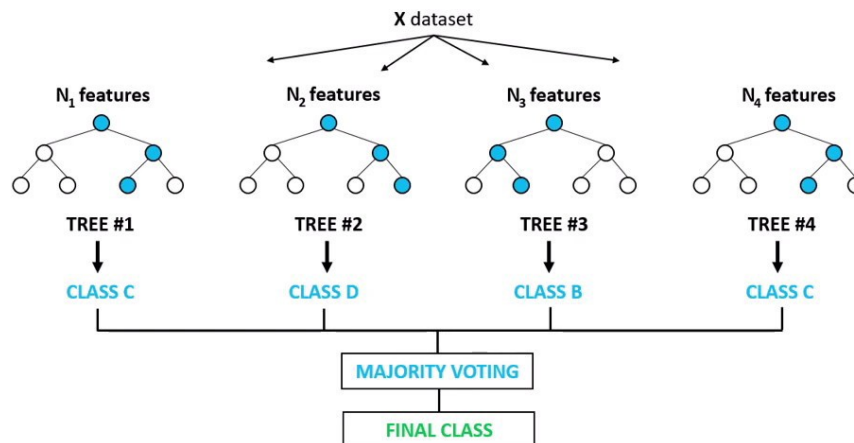


Figure 13: Schematic illustration of a random forest classifier. *Figure from [medium.com](#)*

decision tree are often employed. However, this may lead to suboptimal performance. The next algorithm overcomes partially this issues by combining many decision trees simultaneously.

2.3.2 Random Forests

A random forest is formed by an ensemble of decision trees. Instead of training a single decision tree, many different trees are trained simultaneously. The random forest then aggregates the predictions of its constituent trees, typically by averaging their outputs (see Figure 13). The motivation behind this approach, is that different trees may overfit differently. By combining the outputs of multiple trees, the random forest reduces the overfitting effect of individual decision trees.

Two crucial requirements of random forests are that its constituent decision trees should:

1. have a relatively good performance (better than random guess);
2. be different from each other, so that they compensate their individual errors.

The second condition is satisfied by randomizing the training process of the decision trees, thus the name random forests. The most typical methods to randomize the training process are:

- *Random selection of the training data for each decision tree.* Consider a random forest formed by a set of decision trees $\{\mathcal{T}_1, \dots, \mathcal{T}_F\}$. Given a data set $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^N$, we assign to each tree $\mathcal{T}_j$ a training set $\mathcal{X}_j$ which has been randomly sampled from the original training set $\mathcal{X}$. Usually, $\mathcal{X}_j$ is a bootstrap sample (sample with replacement) of $\mathcal{X}$.
- *Random selection of the features used at each decision split.* If each node has access to all features, the individual trees will have more discriminative power and will fit better the data. However, this can lead to trees making similar decisions, increasing their correlation. On the other hand, if only a subset of features is randomly selected for each decision split, the trees will be more different from each other. However, each individual tree may

become more complex to compensate for the reduced set of features, in order to fit the data properly.

Random forests are a particular instance of a method, called **bagging**, that combines different models. Other techniques to combine different models are boosting and stacking, though we will not study them.

Algorithm 2: Random Forest Training Algorithm

```

Input:  $\mathcal{X} = \{x_i, y_i\}_{i=1}^N$  // Data
Output:  $\mathcal{F}$  // Random Forest
1 for  $i \in 1, \dots, F$  do
2    $\mathcal{X}_i = \text{bootstrap\_sample}(\mathcal{X})$  // Create bootstrap sample of  $\mathcal{X}$ 
   // train tree  $\mathcal{T}_i$ 
3   repeat
4      $\hat{\mathbb{F}} = \text{Subsample}(\mathbb{F})$  // Sample subset features
5      $\text{Split}(\mathcal{T}, \hat{\mathbb{F}}, t)$  // Find best split at node  $t$  with respect to  $\hat{\mathbb{F}}$ 
6   until  $\mathcal{T}_i$  stopping criterium

```

2.3.3 k -Nearest Neighbor classifier

The k -Nearest Neighbor (kNN) classifier is one of the simplest algorithms to train. Given a training set, the kNN algorithm assigns to a query data instance the most present class among its k -nearest neighbors in the training set:

$$kNN(q) = \arg \max_{c \in \mathcal{Y}} \left| \{(x, y) \in \mathcal{N}_k(q) : y = c\} \right|,$$

where $\mathcal{N}_k(q)$ indicates the set of the k -nearest neighbors of q with respect to some similarity/dissimilarity measure. Note that the kNN algorithm does not involve any “learning” process but simply memorizes the training data

The kNN is characterized by two factors:

- *The similarity/dissimilarity measure.* The kNN algorithm assumes that samples closer in this measure should belong to the same class. Therefore, it's crucial that the chosen similarity/dissimilarity measure satisfies this assumption. Nonetheless, when no clear metric exists and the data lies in $\mathbb{R}^m$, the default option is to use the euclidean metric. Note that in this case it is important that all features have the same scale, i.e., it is important that our data is normalized so all the features contribute equally. In Figure 15 we can observe two application of the kNN classifier with and without normalization. When the data is not normalized, the decision of the kNN classifier is principally determined by the variable with smaller range (x_1), which leads to an incorrect classification. Once the data is normalized, both features have similar range making that both features contribute equally to the decision.
- *The number of neighbors k .* The choice of k is crucial to balance between overfitting and underfitting. If k is small (e.g. 1) then we have a perfect classification on the training set,

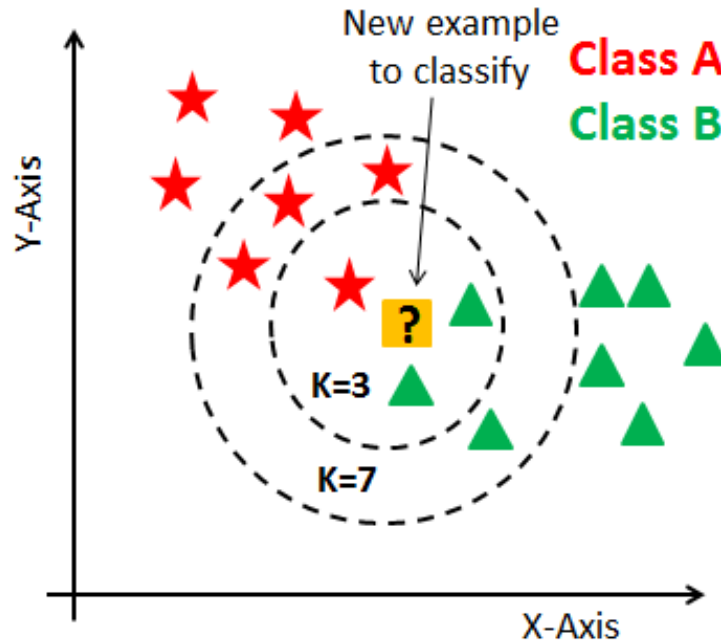


Figure 14: Schematic illustration of the kNN classifier algorithm. For $k = 3$ the unclassified sample would be assigned to green class, while for $k = 7$, it would be assigned to the red one. Figure from [MÄjller and Guido \[2016\]](#).

but the prediction may be too noisy on the test set (overfitting). On the other hand, if k is too large the k -neighborhood of the data may include points that are relatively far and belong to different classes causing misclassifications (underfitting).

The main drawback of the kNN algorithm is its computational complexity, which scales with the number of training samples.

2.3.4 Logistic Regression

Logistic regression is an instance of a linear model. Linear models learn a hyperplane that divide the space, $\mathbb{R}^m$, where the data lies in two regions. This hyperplane is modeled by its normal vector and a bias term represented by $w \in \mathbb{R}^m$ and $b \in \mathbb{R}$. Given a data point x , the classification function is given by

$$f(x) = \begin{cases} 1 & \text{if } \langle w, x \rangle + b \geq 0 \\ 0 & \text{otherwise} \end{cases},$$

where $\langle \cdot, \cdot \rangle$ represents the inner product.

Different linear models differ from each other in the way they measure how well the hyperplane separates the data classes. Given a dataset $\mathcal{X} = \{(x_1, y_1), \dots, (x_n, y_n)\}$, with $y_i \in \{0, 1\}$, logistic regression aims to minimize the following cost function with respect to w :

$$\sum_{x_i} y_i \log(\phi(\langle w, x_i \rangle)) + (1 - y_i) \log(1 - \phi(\langle w, x_i \rangle))$$

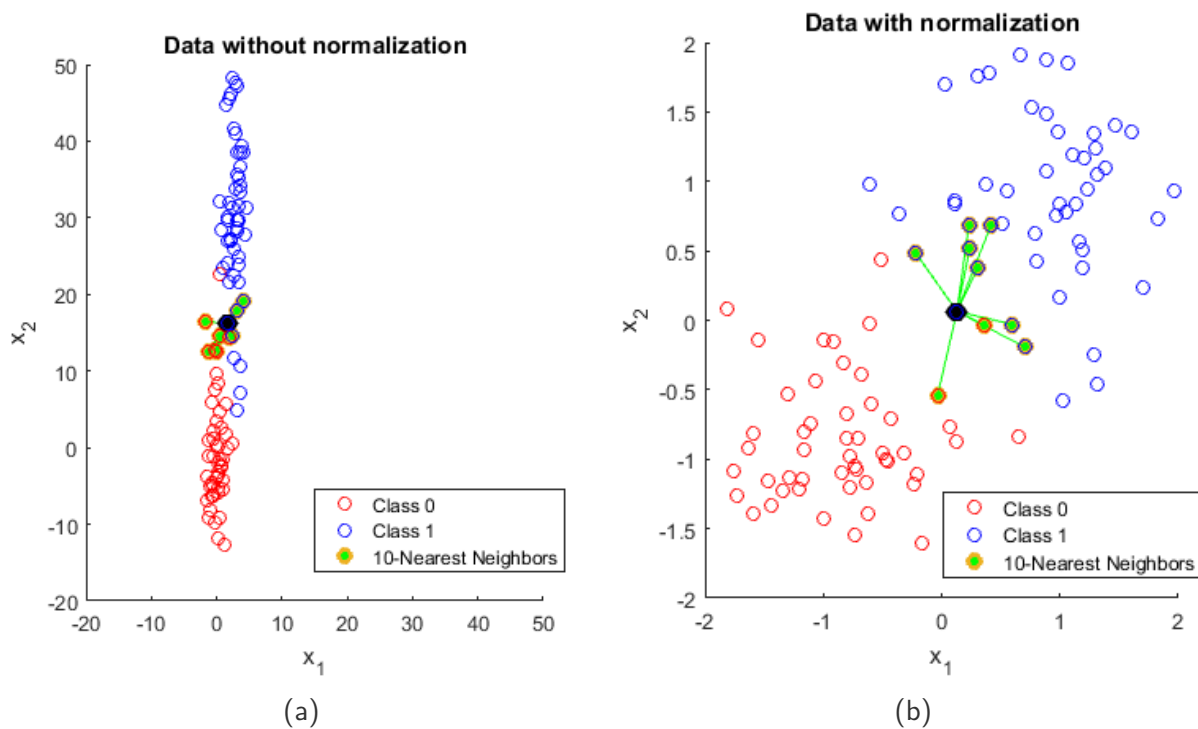


Figure 15: Data normalization importance. kNN classifier applied to a non-normalized data 15a, and normalized data 15b. If the data is non-normalized, the kNN decision relies mostly on the variable of smaller range, x_1 . Once the data is normalized, both features contribute equally, which corrects the missclassification. *Figure from stackoverflow.com*

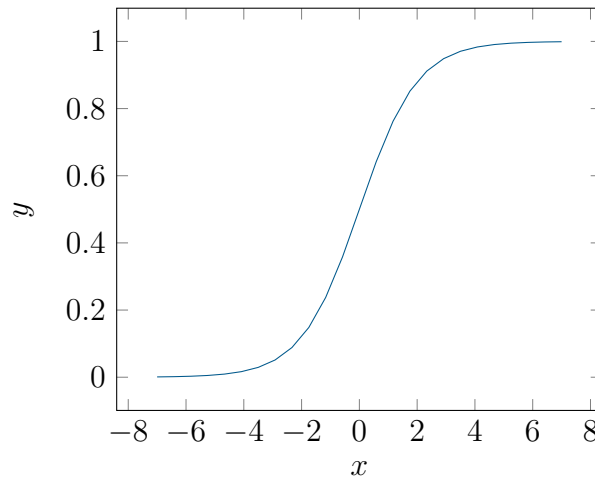


Figure 16: Sigmoid function.

where $\phi(z) = \frac{1}{1+\exp(-z)}$ is the sigmoid function, (see Figure 16). This cost function is known as the cross-entropy loss. Usually $\phi(\langle w, x_i \rangle)$ is interpreted as the probability that sample x_i has label 1, thus logistic regression predicts $p(y_i = 1|x_i)$. Since the output is a continuous probability, logistic regression can be viewed as a regression model for likelihood probabilities, hence its name. Classification is achieved by thresholding the predicted probability $p(y_i = 1|x_i) = \phi(\langle w, x_i \rangle)$, such that

$$f(x_i) = \begin{cases} 1 & \text{if } \phi(\langle w, x_i \rangle) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}.$$

Weight regularization with L2 and L1 norms can be important considerations in logistic regression to prevent overfitting and improve generalization performance. Regularization involves adding a penalty term to the cost function to discourage large weights.

In L2 regularization, also known as Ridge regression, the penalty term is the square of the L2 norm of the weight vector, $\|w\|_2^2$. This encourages smaller weights by adding a term proportional to the sum of the squares of the weights to the original cost function. The modified cost function becomes:

$$\text{Cost}(w) = \sum_i y_i \log(\phi(\langle w, x_i \rangle)) + (1 - y_i) \log(1 - \phi(\langle w, x_i \rangle)) + \lambda \|w\|_2^2$$

where λ is the regularization parameter controlling the strength of regularization.

Similarly, L1 regularization, also known as Lasso regression, adds the L1 norm of the weight vector, $\|w\|_1$, as the penalty term. This promotes sparsity in the weight vector, effectively performing feature selection by driving some weights to zero. The modified cost function becomes:

$$\text{Cost}(w) = \sum_i y_i \log(\phi(\langle w, x_i \rangle)) + (1 - y_i) \log(1 - \phi(\langle w, x_i \rangle)) + \lambda \|w\|_1$$

where again, λ controls the strength of regularization.

By incorporating these regularization terms into the cost function, logistic regression not only learns to classify the data but also aims to keep the weights small, thus preventing overfitting and improving the model's ability to generalize to unseen data.

Remark 2.2 *Though the logistic regression is a binary classifier, it can be generalized to multiclass classification using the so-called technique '1 vs all', i.e., by learning a separating hyperplane for each class. The classification is obtained by assigning the label with the highest score.*

3 Unsupervised learning

Unsupervised learning is the type of learning where we aim to explore and understand the data without any kind of guidance. We will focus on two unsupervised learning tasks: clustering and dimensionality reduction.

3.1 Clustering

Clustering is the task of partitioning the data into different sets such that elements of the same cluster are more similar, under certain criteria, in comparison to data points of other clusters. Clustering is closely related to the classification task. We also aim to group the data points into different classes/clusters, but in contrast to the classification task, we do not have access to the target labels.

A set can be partitioned according to different criteria. For instance, consider Figure 17. On the one hand, we could argue that there are two clusters, one containing two images of dogs, and the other containing two images of cats. On the other hand, we could also distinguish two clusters if our criterion was based on the style of the images and not on the content. In this case, we would have a cluster composed of images in black and white, and a second cluster with images in color. Since we do not have access to the ground truth labels, we can not claim a priori that one cluster is better than other. Nonetheless, depending on the dataset we may know that some criteria may be appropriate than others.

The selection of a clustering algorithm depends on our prior understanding of the data, as different algorithms embody distinct inductive biases, resulting in different partitioning criteria. In this section, we will explore three clustering algorithms: k -means, agglomerative (linkage) clustering, and DBSCAN.

3.1.1 k -Means

The k -means algorithm is an iterative algorithm that partitions the data into a predefined number of clusters k . The membership of the data points to the clusters is determined by the distance to the representatives of the clusters. The representative of a cluster in the k -means algorithm coincides with the center (mean) of the cluster. An element will be assigned to the cluster whose representative is closest with respect to some dissimilarity measure.

At the beginning of the algorithm, k data points are set to be the k cluster representatives or centroids. Different techniques exist to select the initial k cluster centroids. The most simple one is to choose them randomly from the set of points. A popular alternative is the so-called



(a) *Source*



(b) *Source*



(c) *Source*



(d) *Source*

Figure 17: Given these pictures, different criteria can be used to partition the images. With respect to the style of the image we could differentiate between black and white and color images. If we consider the content of the pictures only, we would divide the pictures into two clusters formed by pictures of dogs and cats respectively.

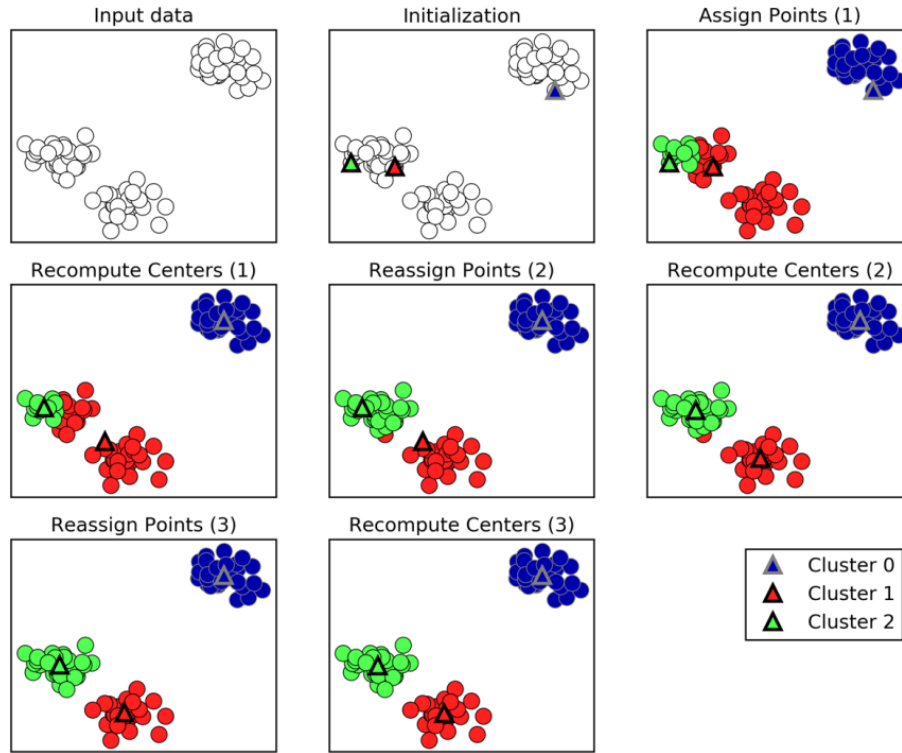


Figure 18: k -means algorithm steps. *Figure from MÄijller and Guido [2016]*

“ k -means++”, which selects iteratively a new centroid among the data points prioritizing the ones which are further away from the already selected centroids (see Algorithm 4).

Once the centroids have been initialized, k -means performs two steps till convergence:

- **Data assignment:** Each data point is assigned to the cluster whose centroid is closest. Ties are broken arbitrarily:

$$\text{Assignment}(x) = \arg \min_{c \in \mathcal{C}} (d(x, c)), \quad (3)$$

where $d(\cdot, \cdot)$ is the dissimilarity measure and $\mathcal{C}$ is the set of centroids.

- **Centroid update:** The centroids of each cluster C are relocated to the center (mean) of all the data points that have been assigned to C in the previous step.

$$\text{UpdateCentroid}(C) = \arg \min_x \sum_{y \in C} d(x, y)^2, \quad (4)$$

When $d(\cdot, \cdot)$ is equal to the euclidean distance, then the centroid coincides with the mean of the cluster.

$$\text{UpdateCentroid}(C) = \arg \min_x \sum_{y \in C} \|x - y\|^2 = \frac{1}{|C|} \sum_{y \in C} y$$

This algorithm is due to Lloyd. A visualization of its execution is depicted in Figure 18 and the pseudocode can be found in Algorithm 3.

The k -means aims to optimize the following objective function

$$\min_{\mathcal{C} : |\mathcal{C}|=k} \sum_{x \in X} \min_{c \in \mathcal{C}} (d(x, c))^2 \quad (5)$$

The function is minimized over the set with cardinality k of possible of cluster centroids. The default dissimilarity measure $d(\cdot, \cdot)$ is the euclidean distance, though others can be used. The optimization objective (5) is not only non-convex, but its optimization is NP-hard. The greedy nature of Lloyd's algorithm together with the non-convexity of the objective function implies that the convergence is only to a local minima, which causes that the algorithm is sensitive to initialization. However, the local optimum problem can be mitigated by running multiple times the algorithm.

Another limitation of the algorithm is its sensitivity to outliers, since it relies on the mean statistic, which is also is sensitive to them.

Moreover, the k -means algorithm with the euclidean norm performs well when the clusters are spherical and well separated, but may fail with non-convex data shapes. Indeed, the euclidean norm is isotropic and favours all directions equally, which may be detrimental when the data is not spherical. Nonetheless, the use of alternative dissimilarity measures can alleviate this issue.

Algorithm 3: k -means algorithm

Input: $\mathcal{X} = \{x_i\}_{i=1}^N$ // Data
Output: Clustering
1 $\mathcal{C} = \text{kmeans}++(\mathcal{X})$ // Initialize centroids
2 **repeat**
3 Clusters = Update_clusters(Clusters, $\mathcal{C}$, $\mathcal{X}$)
 $\mathcal{C} = \text{Update_centroids}(\text{Clusters}, \mathcal{C}, \mathcal{X})$
4 **until** Convergence of Clusters

Algorithm 4: k -means++ initialization

Input: $\mathcal{X} = \{x_i\}_{i=1}^N$ // Data
Output: $\mathcal{C}$ // Dinitial k centroids
1 $c = \text{sample}(\mathcal{X})$ // sample randomly 1st centroid
2 $\mathcal{C}.\text{append}(c)$ // Add centroid to list of centroids
3 **repeat**
4 **for** $x \in \mathcal{X}$ **do**
5 $d_x = d(x, \mathcal{C})$ // Compute distance to closest cluster
6 $p_x \propto d_x^2$ // assign probabiltiy proportional to d_x^2
7 $c = \text{sample}(\mathcal{X}, p_{\mathcal{X}})$ // sample new centroid c with probability p_c
8 **until** number of centroids equals k

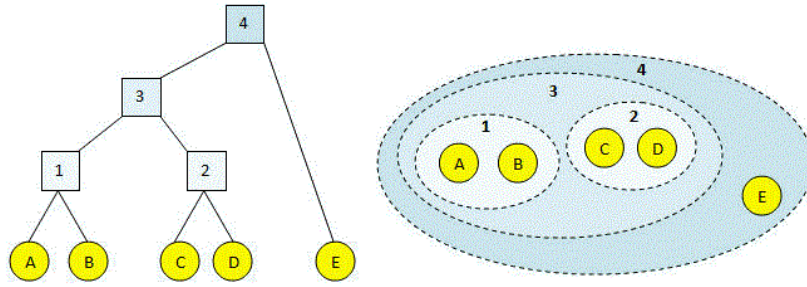


Figure 19: Figure from analyticsvidhya.com

3.1.2 Agglomerative clustering

Hierarchical agglomerative clustering is a family of clustering algorithms which define different partitions at different levels in a hierarchical fashion. The output of these algorithms can be represented as a tree, called dendrogram (see Figure 19), which describes the hierarchical structure of the clustering. Each node of the dendrogram represents a cluster at a different level. The root of the tree represents the partition with a single set (all elements form a unique cluster). Two nodes (clusters) in the dendrogram, C_1 and C_2 , are connected if and only if, one of them is a subset of the other, say $C_1 \subset C_2$, and there does not exist any other cluster, $\hat{C}$, which satisfies $C_1 \subset \hat{C} \subset C_2$. The set of leaves of the dendrogram, represent the partition clustering where each data sample forms its own cluster. By cutting this tree at different heights, we can induce a different number of (nested) clusters.

All agglomerative clustering algorithms start by setting each data point as its own cluster. The input of the agglomerative clustering is a symmetric $N \times N$ dissimilarity matrix, such that $D_{ij} = d(i, j) > 0$, where d is a non-negative dissimilarity measure and N is the number of data samples. At each iteration, the agglomerative algorithm merges the two most similar clusters. See Algorithm 5 for the pseudocode and Figure 20 for a visualization of the steps. In some cases, generating the entire dendrogram of clusters may not be necessary, and we may only be interested in a single partition. In such instances, the algorithm can be terminated earlier using a stopping criterion, such as specifying the desired number of clusters or setting a threshold on the minimum distance to merge two clusters.

The dissimilarity between the clusters, $d_C(\cdot, \cdot)$, is generalized from the dissimilarity between the data points. Different algorithms apply distinct generalizations. Next, we will describe some of these approaches and how do they define the dissimilarity between the clusters:

- **Single linkage clustering:** The closeness of two clusters is measured by the minimum of the pairwise dissimilarities between the points of both clusters, i.e.

$$d_{SL}(C_1, C_2) = \min_{i \in C_1, j \in C_2} d(i, j). \quad (6)$$

- **Complete linkage clustering:** The dissimilarity between two clusters is measured by the maximum of the pairwise dissimilarities between the elements of both clusters.

$$d_{CL}(C_1, C_2) = \max_{i \in C_1, j \in C_2} d(i, j). \quad (7)$$

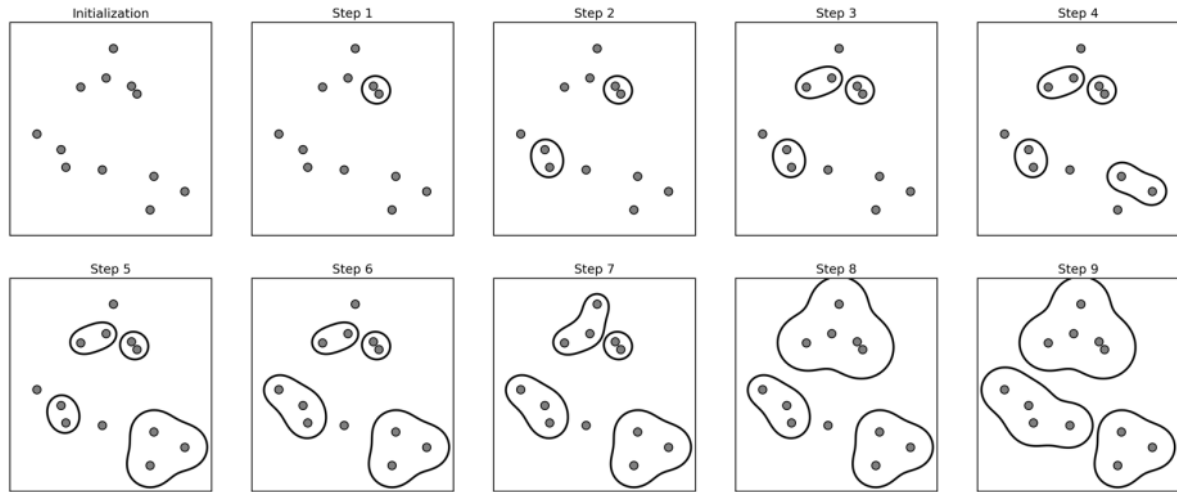


Figure 20: Agglomerative clustering iteratively joins the two closest clusters. *Figure from MÄijller and Guido [2016]*

Algorithm 5: Agglomerative clustering

Input: D // Dissimilarity matrix
Output: dendrogram

```

1 for  $i \in \{1, \dots, N\}$  do
2    $C_i = \{i\}$  // Initialize clusters as singletons
3  $S = \{1, \dots, N\}$  // Initialize set of clusters available for merging
4  $D_C \leftarrow D$  // Set data dissimilarity equal to cluster dissimilarity
5 repeat
6    $(j^*, k^*) \leftarrow \arg \min_{j, k \in S} d_C(j, k)$  // select most similar cluster pair
7    $C_\ell \leftarrow C_{j^*} \cup C_{k^*}$  // Create new cluster
8    $S \leftarrow S \setminus \{j^*, k^*\}$  // Remove  $j^*$  and  $k^*$  from available clusters
9   if  $\ell \notin S$  then
10     $S \leftarrow S \cup \{\ell\}$  // Mark  $\ell$  as available
11   for  $i \in S$  do
12     $D_C \leftarrow d_C(i, \ell)$  // Update cluster dissimilarity matrix
13 until no more clusters are available for merging
  
```

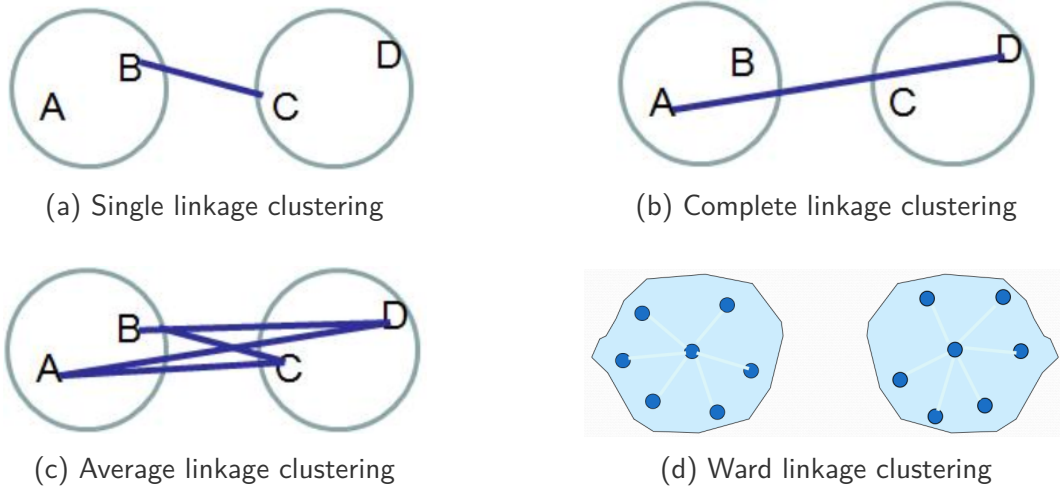


Figure 21: Figures 21a, 21b and 21c from [Murphy \[2022\]](#), and Figure 21d from [Source](#).

Complete linkage clustering prioritizes compact clusters by penalizing those with high diameters. This is achieved by merging the two clusters whose union minimizes the diameter. In contrast, single linkage clustering only requires that a single pair of objects is close, potentially resulting in clusters with large diameters. As a result, complete linkage clustering tends to produce more compact clusters compared to single linkage clustering.

- *Average linkage clustering*: It defines the cluster dissimilarity as the average pairwise dissimilarity between the elements of both clusters.

$$d_{CL}(C_1, C_2) = \frac{1}{|C_1||C_2|} \sum_{i \in C_1, j \in C_2} d(i, j). \quad (8)$$

Average linkage clustering represents a compromise between single and complete linkage clustering. It tends to produce relatively compact clusters that are relatively far apart.

- *Ward linkage clustering*: It merges the clusters that result in the lowest increment of the intra-cluster variance.

$$d_{WL}(C_1, C_2) = \sum_{i \in C_1 \cup C_2} d(i, m(C_1 \cup C_2))^2 - \sum_{i \in C_1} d(i, m(C_1))^2 - \sum_{i \in C_2} d(i, m(C_2))^2 \quad (9)$$

where $m(C)$ is the center of the cluster. If $d(\cdot, \cdot)$ is the euclidean distance then, the computation reduces to

$$d_{WL}(C_1, C_2) = \frac{|C_1||C_2|}{|C_1| + |C_2|} \|m(C_1) - m(C_2)\|^2, \quad (10)$$

where $m(C) = \frac{1}{|C|} \sum_{i \in C} i$.

Figure 22, obtained from the experiment run by [scikit-learn.org](#), compares the different agglomerative algorithms.

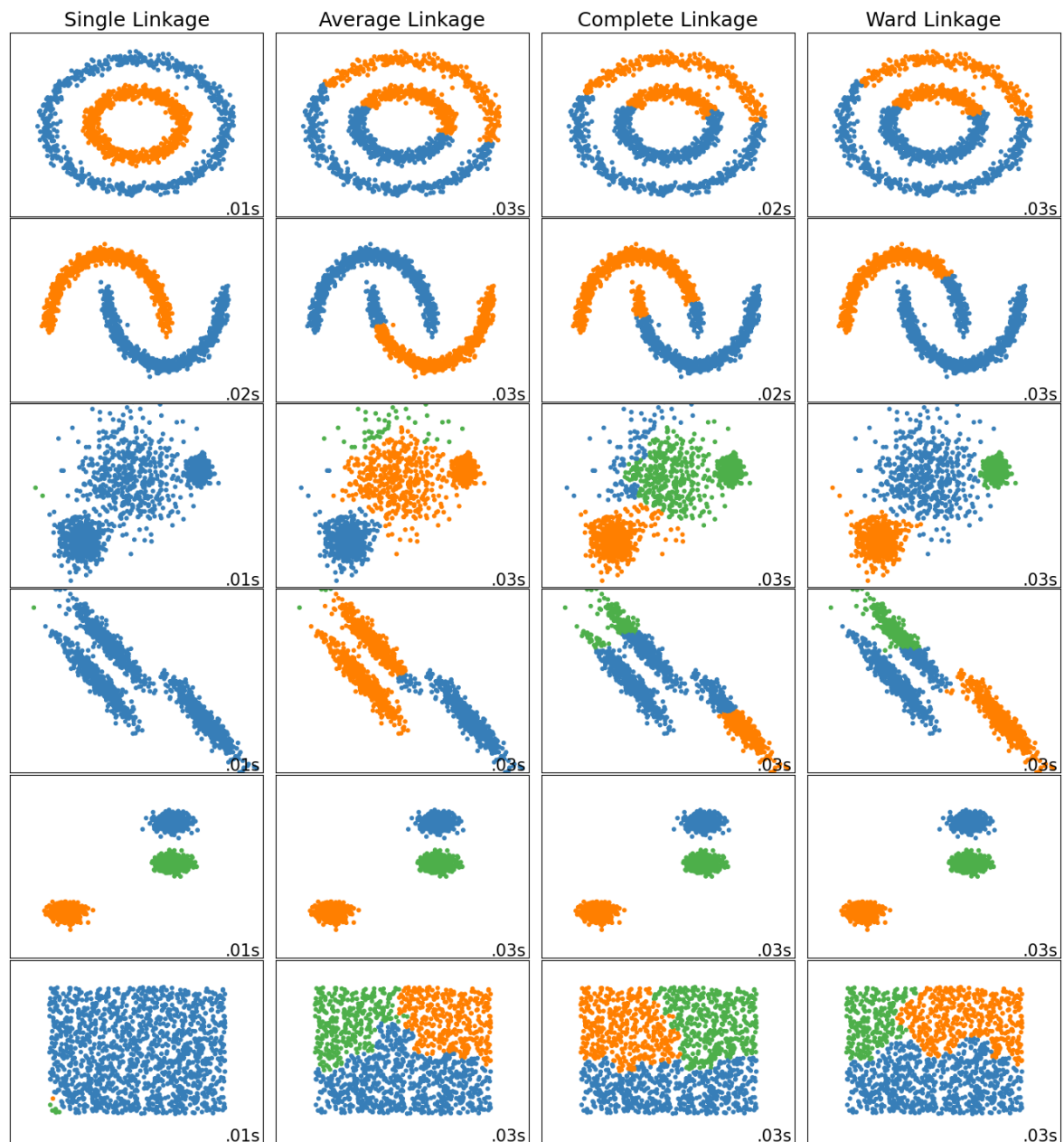


Figure 22: Comparison between different agglomerative algorithms. Single linkage performs well on non-globular data, but it performs poorly in the presence of noise. Average and complete linkage perform well on cleanly separated globular clusters, but have mixed results otherwise. Ward is the most effective method for noisy data. *Figure from scikit-learn.org*

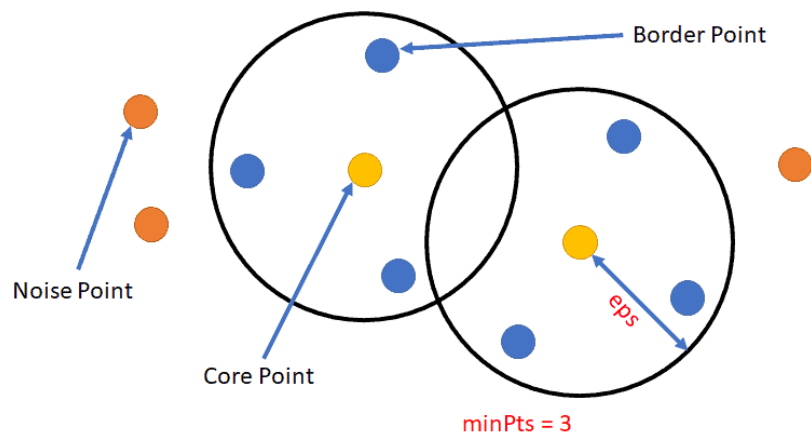


Figure 23: DBSCAN differentiates 3 types of data points: Core, Border and Noise points. Figure from machinelearninggeek.com

3.1.3 DBSCAN

"Density based spatial clustering of applications with noise" (DBSCAN) is an algorithm that defines the clusters based on their density. DBSCAN works on the assumption that clusters are dense regions separated by low density regions. This algorithm does not require as input the number of clusters and can identify noise points that do not belong to any cluster. The algorithm identifies three types of data points: core, border and noise points. This classification is based on the following parameters, which ultimately also determine the final clustering (see Figure 24):

- ϵ : radius of the ball generated around each point. This parameter determines what it means for a point to be close to another point.
- **minPts**: Minimum number of samples contained inside the ball of a point to be classified as a dense point (core point).

A metric or dissimilarity measure is also necessary to determine which points are within ϵ distance of a given data sample.

The three types of points are defined as follows (see Figure 23):

- **Core point**: As already mentioned, a core point is a point, whose associated ball of radius ϵ centered at this particular data point contains at least **minPts**.
- **Border point**: Points that have less than **minPts** in its associated ball of radius ϵ but do contain a core point in it will be called border points.
- **Noise point**: If a point is not a core point neither a border point, then we consider it as noise.

Consider the ϵ -graph, where two nodes (points) are connected by an edge if the distance between them is lower than ϵ . In DBSCAN, two points belong to the same cluster if there exists a path in the ϵ -graph connecting them, such that all intermediate points on the path

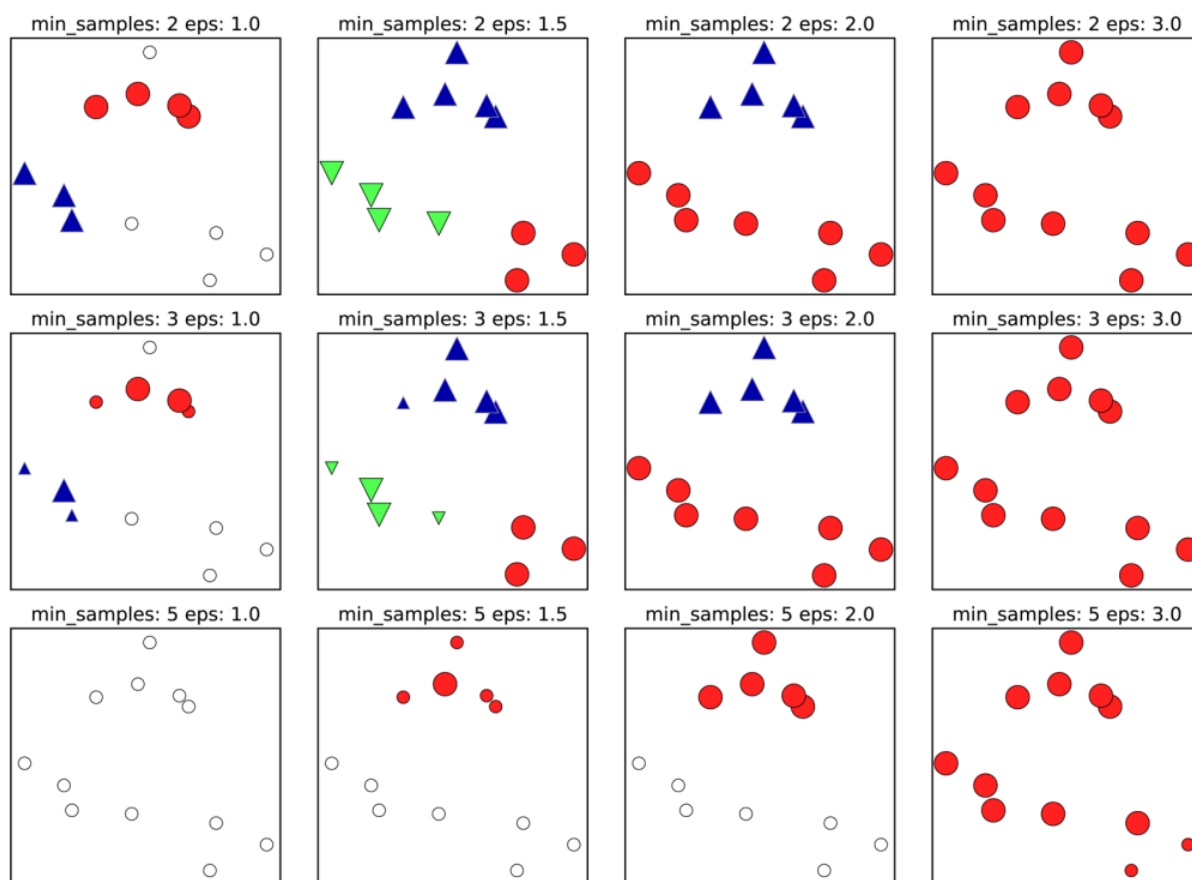


Figure 24: Cluster assignments found by DBSCAN with varying settings for the `minPts` (`min_samples`) and ϵ (`eps`) parameters. Different clusters are represented by different color and shape. Core points are shown as larger markers while border points are displayed as smaller markers. Noise points are colorless. *Figure from Mäijler and Guido [2016].*

are core points. If the path consists of only two points, one of them must be a core point for them to be considered part of the same cluster. Thus, in DBSCAN the ϵ -neighborhood of a core point must belong to the same cluster as the core point itself, and two core points are in the same cluster if their neighborhoods overlap. Algorithm 6 describes the pseudocode for DBSCAN.

Algorithm 6: DBSCAN

- 1 Identify core, border and noise points
 - 2 Form a separate cluster for each core point or group of connected core points
 - 3 Assign each border point to the cluster of its corresponding core point
-

DBSCAN is a robust and yet simple algorithm that can detect clusters of arbitrary shape. However, an obvious drawback is that it has two parameters ϵ and `minPts` for which choosing good values may be challenging in some cases.

3.1.4 Performance measures

Assessing the quality of clusterings is challenging due to the absence of ground truth labels. However, leveraging prior knowledge about the data can aid in evaluating algorithm performance. Three commonly used quality measures are the Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), and the Silhouette Score.

Adjusted rand index (ARI): This metric assesses the similarity between two clusterings, particularly when a ground truth or reference partition (C_R) is available for comparison with the proposed clustering (C_P).

To compute the ARI, we first calculate the Rand Index (RI), which quantifies the agreement between C_P and C_R . The RI considers pairs of points and counts those that are either clustered together or separately in both partitions. Specifically:

- a : Number of pairs of elements in the same cluster in both C_P and C_R .
- b : Number of pairs of elements in different clusters in both C_P and C_R .
- c : Number of pairs of elements in the same cluster in C_P but different clusters in C_R .
- d : Number of pairs of elements in different clusters in C_P but the same cluster in C_R .

The RI is then computed as

$$RI = \frac{a + b}{a + b + c + d}, \quad (11)$$

where $a + b$ represents agreements and $c + d$ represents disagreements. Thus, RI can be considered as the frequency of occurrence of agreements over the total of pairs. The ARI is the corrected-for-chance version of the RI, which considers as a baseline the expected similarity of all pair-wise comparisons between partitions specified by a random model. Formally, the ARI measure computes

$$ARI = \frac{RI - \text{Expected}(RI)}{\max(RI) - \text{Expected}(RI)}. \quad (12)$$

While the Rand Index (RI) ranges between 0 and 1, where a value of 1 signifies perfect agreement between the proposed and reference clusters, the Adjusted Rand Index (ARI) extends its range from -1 to 1. A positive ARI indicates agreement beyond what would be expected by chance, with 1 representing perfect agreement. Conversely, a negative ARI suggests a lower RI score than expected under a random model, indicating poor clustering performance relative to chance agreement.

Normalized mutual information (NMI): Similar to the ARI measure, NMI also requires access to a reference clustering. NMI quantifies the normalized mutual information between two cluster assignments, measuring the shared information between them. Intuitively, NMI evaluates how well one clustering predicts the other and vice versa, with higher values indicating greater similarity between the two clusterings.

Formally, NMI treats each partition, C , as a probability distribution over the clusters, where the probability of a cluster is proportional to its cardinality. i.e. $p(c) \propto |c|$ for $c \in C$. The

NMI is computed using the mutual information (MI) between the cluster assignments, defined as the difference between the joint entropy and the conditional entropies of the partitions. Specifically, NMI is given by the formula:

$$\text{NMI} = \frac{2I(C_R; C_P)}{H(C_R) + H(C_P)}, \quad (13)$$

where H is the entropy and I is the mutual information with formula

$$H(C) = - \sum_{c \in C} p(c) \log(p(c))$$

$$I(C_1; C_2) = H(C_1, C_2) - H(C_1|C_2) - H(C_2|C_1).$$

The terms $H(C_1|C_2)$ and $H(C_1, C_2)$ refer to the entropy of the conditional distribution and joint distribution of C_1 and C_2 . The NMI score ranges from 0 to 1, with higher values indicating greater similarity between the clusterings. A value of 1 signifies that the clusterings are identical.

Silhouette score: The Silhouette score is a metric used to calculate the goodness of a clustering technique. Unlike ARI and NMI, the Silhouette score doesn't require ground truth labels. Instead, it evaluates the consistency within clusters and the separation between clusters based on the distances between data points.

To compute the Silhouette score for each sample, we first calculate two distances:

- The mean distance between a sample and all other points in the same cluster, denoted as $a(x)$

$$a(x) = \frac{1}{|c_i| - 1} \sum_{y \in c_i} d(x, y)$$

- The mean distance between a sample and all points in the nearest cluster that the sample is not a part of, denoted as $b(x)$

$$b(x) = \min_{i \neq j} \frac{1}{|c_j|} \sum_{y \in c_j} d(x, y)$$

The Silhouette score ($s(x)$) for each sample is then defined as:

$$\text{silhouette} = \frac{b - a}{\max(a, b)} = \begin{cases} 1 - a(x)/b(x), & \text{if } a(x) < b(x) \\ 0, & \text{if } a(x) = b(x) \\ b(x)/a(x) - 1, & \text{if } b(x) < a(x) \end{cases} \quad (14)$$

The overall Silhouette score of the clustering is the mean of the Silhouette score for all samples. The value of the Silhouette score ranges from -1 to 1. A higher Silhouette score suggests better-defined clusters with distinct separation (≈ 1), while a lower score indicates overlapping clusters (≈ 0) or misclassifications (≈ -1). The Silhouette score provides valuable insight into the compactness and separation of clusters, aiding in the evaluation and selection of clustering algorithms.

3.2 Dimensionality Reduction

In many problems the data gathered is represented in a high dimensional space, which may have thousands or even millions of dimensions. In addition to increase the computational cost of the data processing, such a high dimensional representation may difficult the discovery of good solutions or models. In part, this is a consequence of the so called **curse of dimensionality**. Briefly, the curse of dimensionality indicates that the higher the dimension of the input space, more samples will be required to approximate a function with a given guaranteed accuracy. More specifically, the number of samples increases exponentially with the number of dimensions. In our case, the dimension of the data is given by the dimensionality of the features.

Fortunately, in some problems not all features are relevant to describe the data. Dimensionality reduction aims to find an alternative representation of the data in a lower dimensional space which is much more efficient describing the data, at the cost of some data loss (this data loss may also be noise). We will only study two dimensionality reduction techniques: *Principal Component Analysis* (PCA) and *t-distributed stochastic neighbor embedding* (t-SNE).

3.2.1 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is the most popular dimensionality reduction algorithm. PCA projects linearly the data from a high dimensional space onto a lower dimensional hyperplane. From all possible hyperplanes, PCA projects it onto the one that preserves the maximum variance. Briefly, PCA selects the axes of higher variance (principal components) and projects the data onto these axes. Figure 25 depicts the projection of a two dimensional dataset onto different lines. Intuitively, projecting onto the axis that preserves the maximum variance appears reasonable, as it minimizes the potential loss of information compared to other projections. Another argument in favour of the PCA projection is its role in minimizing the mean squared distance between the original dataset and its projection. In the following, we will explain briefly how to compute the principal components (see Algorithm 7 for pseudocode). For a comprehensive theoretical foundation and detailed derivations we refer to Bishop [2006]

Let $X \in \mathbb{R}^{N \times n}$ be the data matrix, where each row encodes the features of a sample of the dataset, i.e. a data sample lies in $\mathbb{R}^n$. Additionally, let us assume without loss of generality, that the samples are centered. That is, the mean of all samples is the 0 vector³. As mentioned earlier, PCA projects linearly the data points into a lower dimensional hyperplane. PCA learns a transformation matrix $W^{n \times d}$, where $d < n$, such that the transformation

$$xW = z,$$

maps each n -dimensional sample x onto its associated d -dimensional projection z . The columns of matrix W consist of the top d eigenvectors of the covariance matrix of the centered data, $\Sigma = XX^\top$.⁴ Thus, one can reduce the dimensionality of the dataset down to d dimensions by projecting it onto the hyperplane defined by the first d eigenvectors of $\Sigma = XX^\top$.

³Any data can be centered by translating in the opposite direction of the mean, $\bar{X}$.

⁴Technically, the covariance matrix is equal to $\frac{1}{N}(X - \bar{X})(X - \bar{X})^\top$. Nonetheless, since we are assuming X centered, we have $\bar{X} = 0$ and therefore we can ignore it. We also ignore the factor $\frac{1}{N}$ because it does not have any effect on the eigenvectors.

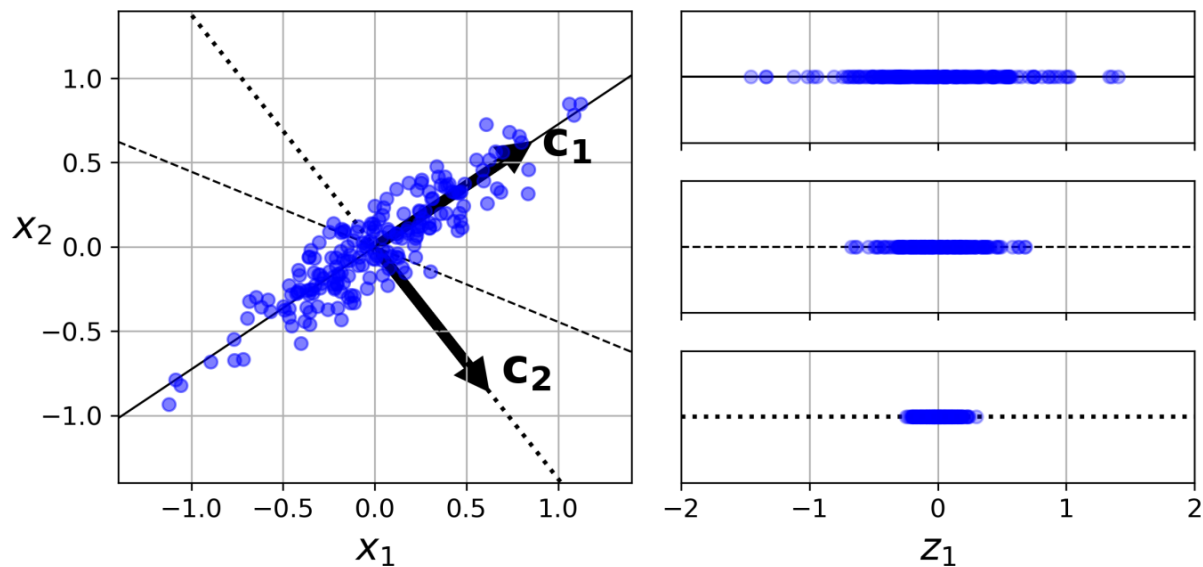


Figure 25: PCA selects the hyperplane which preserves the maximum variance. A dataset in two dimensions is projected onto different lines (hyperplanes of dimension 1). The dotted line preserves very little variance, the dashed line preserves an intermediate amount of variance. The continuous line, which coincides with the first principal axis (C_1), preserves the maximum variance. *Figure from GÄrron [2019].*

Note that we assume that the eigenvectors have norm 1. Moreover, since the covariance matrix is symmetric, we can ensure that the set of eigenvectors are orthogonal to each other.

As mentioned earlier, PCA projects onto the axis that preserve the maximum amount of variance. The variance in the direction of a principal component is given by the eigenvalue associated with it. Given the eigenvalues $\{\lambda_k\}$, the fraction of the variance explained by the i -th principal component is given by

$$\frac{\lambda_i}{\sum_k \lambda_k}.$$

Remark 3.1 Note that the eigenvectors of the covariance matrix, even when normalized to unit length, are not unique. If an eigenvalue λ_i has multiplicity 1, the eigenspace has dimension 1, and therefore it is spanned by a single vector. However, if eigenvector v spanning the eigenspace has norm 1, the vector $-v$ will also have norm 1. Thus, depending on the algorithm used, we may obtain different projections, which will be reflections of each other with respect to the origin.

When the eigenvalue λ_i has multiplicity greater than 1, we can find an infinite number of orthogonal eigenvectors spanning the eigenspace. In this case, the difference between the projections will account for rotations and reflections corresponding to different choices of orthogonal bases for the eigenspace. Nonetheless, in actual data, it is rare for an eigenvalue to have multiplicity greater than 1.

Remark 3.2 Note that the PCA directions are highly sensitive to data scaling. Thus, in case that the features have been measured on different scales and we want to assign equal importance to all features, we will need to standardize the features prior to the application of PCA.

Algorithm 7: PCA

Input: $X \in \mathbb{R}^{N \times n}$, d // Data matrix, dimension projection space
Output: $Z \in \mathbb{R}^{N \times d}$ // PCA projection

- 1 $X \leftarrow \text{Standardize}(X)$ // Standardize data (center and scale data)
- 2 $\Sigma \leftarrow XX^\top$ // Compute covariance matrix
- 3 $W \leftarrow \text{EigenVectors}(\Sigma, d)$ // Get projection matrix from the first d eigenvectors of sigma
- 4 $Z \leftarrow XW$ // Project data

3.2.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-Distributed Stochastic Neighbor Embedding (t-SNE) [Maaten and Hinton \[2008\]](#) is a dimensionality reduction technique that is mostly used **for visualization**. The primary objective of t-SNE is to **reduce the dimensionality of the data while preserving the local structure and relationships among data points**. In essence, t-SNE aims to map similar instances in the original high-dimensional space to nearby points in the lower-dimensional embedding. In contrast to PCA, this is a non-linear dimensionality reduction technique.

Given a dataset $\{x_i\}_{i=1}^N$, t-SNE models two distributions over pairs of nodes, one for the high dimensional embedding and another one for the low-dimensional embedding. The probability of a pair of points in each distribution depends on the euclidean distance between the points in the corresponding ambient space. The high dimensional pairs of points distribution, $P_{i|j}$, builds on the gaussian distribution, while the low dimensional pairs of points distribution, $Q_{i|j}$ is based on the t-distribution (t in t-SNE stands for t-distribution):

$$P_{j|i} = \frac{\exp\left(-\frac{1}{2\sigma_i^2} \|x_i - x_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\frac{1}{2\sigma_i^2} \|x_i - x_k\|^2\right)} \quad (15)$$

$$Q_{j|i} = \frac{(1 + \|z_i - z_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|z_i - z_k\|^2)^{-1}} \quad (16)$$

The values $P_{i|j}$ and $Q_{i|j}$ model the probability that point i would select point j as its neighbor in the high and low dimensional space respectively. The parameter σ_i^2 regulates the scale notion of sample i . In practice, the parameter σ_i^2 is chosen so that the distribution $P_{\cdot|i}$ has a perplexity specified by the user⁵. The goal of the t-SNE algorithm is to find the low dimensional embedding which minimizes the KL-divergence between these distributions

$$\sum_i D_{KL}(P_{\cdot|i} || Q_{\cdot|i}) = \sum_i \sum_j P_{j|i} \log \left(\frac{P_{j|i}}{Q_{j|i}} \right).$$

This objective function is non convex, nevertheless it can be minimized using stochastic gradient descent. Note that t-SNE focuses on points that lie close to each other, rather than preserving distances between far-apart points. Thus, distances between clusters in the t-SNE visualization may not mean anything. We refer to the following [blog](#) for a more applied analysis of t-SNE.

⁵The perplexity of a distribution P is defined as $2^{H(P)}$, where $H(P)$ is the entropy of the distribution

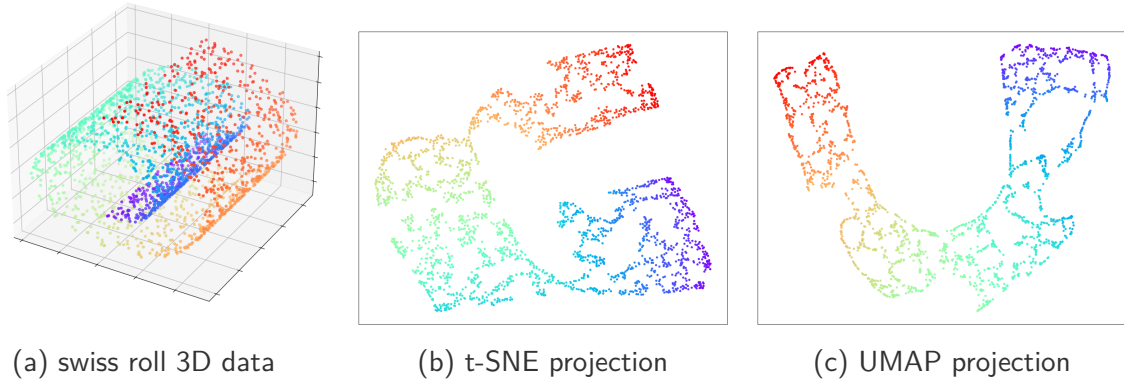


Figure 26: t-SNE and UMAP 2D-visualizations of the swiss roll 3D-data.

3.2.3 Uniform Manifold Approximation and Projection (UMAP)

Similar to t-SNE, Uniform Manifold Approximation and Projection (UMAP) [McInnes et al. \[2018\]](#) is a **non-linear dimensionality reduction technique**. It also aims to map similar instances from the original high-dimensional space to nearby points in a lower-dimensional representation. However, UMAP **differs from t-SNE in how it defines similarities between the samples in both high and low-dimensional spaces**.

Let $\{x_i\}_{i=1}^N$ represent the high-dimensional data samples and $\{z_i\}_{i=1}^N$ the learned embedding in the low dimensional space. UMAP models two distributions for each pair of points: one for the high-dimensional data and one for the low-dimensional representation. The high-dimensional pair distribution, $P_{i,j}$, and the low-dimensional pair distribution, $Q_{i,j}$, are given by:

$$P_{i,j} = w_i(x_i, x_j) + w_j(x_j, x_i) - w_i(x_i, x_j)w_j(x_j, x_i), \quad (17)$$

$$Q_{i,j} = w_i(z_i, z_j) + w_j(z_j, z_i) - w_i(z_i, z_j)w'_j(z_j, z_i), \quad (18)$$

where

$$w_i(x_i, x_j) := \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right), \quad (19)$$

$$w_i(z_i, z_j) := \exp\left(-\frac{d(z_i, z_j) - \rho'_i}{\sigma'_i}\right).$$

Here, $d(\cdot, \cdot)$ represents the Euclidean distance in the relevant space. The values ρ_i and ρ'_i are the distances from x_i and z_i to their closest neighbors, respectively, while σ_i and σ'_i represent the distances from x_i and z_i to their k -th closest neighbors (where k is a hyperparameter of the algorithm).

The values $P_{i,j}$ and $Q_{i,j}$ can be interpreted as fuzzy set memberships, indicating the likelihood that the points are related to each other. UMAP aims to find a low-dimensional embedding that minimizes the KL-divergence between these distributions for all pairs of points, using the following objective function:

$$\sum_{i,j} D_{KL}(P_{i,j} || Q_{i,j}) = \sum_{i,j} P_{i,j} \log\left(\frac{P_{i,j}}{Q_{i,j}}\right) + (1 - P_{i,j}) \log\left(\frac{1 - P_{i,j}}{1 - Q_{i,j}}\right). \quad (20)$$

This objective function is non-convex but can be minimized using stochastic gradient descent. To reduce the computational cost of evaluating all pairs of points, UMAP uses negative sampling. In this approach, the k -nearest neighbors for each point, along with a random subset of non-neighbors, are used to approximate the loss during optimization.

Remark 3.3 As shown in [Damrich and Hamprecht \[2021\]](#), using negative sampling the loss that it is actually being optimized differs from the one defined in (20). Specifically, negative sampling makes UMAP focus primarily on preserving the k -nearest neighbor graph, rather than the exact similarities defined by (19).

Like t-SNE, UMAP emphasizes preserving the relationships between nearby points, rather than accurately reflecting distances between points that are far apart. Consequently, the distances between clusters in UMAP visualizations may not hold meaningful interpretations, just as in t-SNE. For a more practical analysis of UMAP, refer to this [blog](#).

4 Deep learning

In the last decade, deep learning has revolutionized the field of machine learning. In this section we will review some of the very basic neural network architectures and how to train them.

4.1 Network Training

4.1.1 Training

Briefly, a neural network is a function $f(\cdot; w)$ which is parameterized by some parameters called weights, here denoted by w . In this section we will describe how to train a neural network. We will use as a running example the classification task. Note that for the classification task, neural networks output the probability of x being assigned to label y , i.e. it predicts $p(y|x)$. Usually, the probability is modeled via the sigmoid or softmax activation functions (see Section 4.2.3).

Training a neural network involves solving an optimization problem, where the goal is to minimize a loss function

$$\arg \min_{f \in \mathcal{F}} L(\mathcal{X}, f), \quad (21)$$

where $\mathcal{F}$ is a family of functions. In concrete, $\mathcal{F}$ is the family of functions that the neural network can realize with its parameters. In practice, (21) becomes

$$\arg \min_w L(\mathcal{X}, f(\cdot; w)), \quad (22)$$

where we minimize over the weights of the network. L is a loss function which measures how well is the task solved by the network. For instance, a typical loss function for the classification task is the cross entropy loss, which is the KL divergence between the network predicted class probability distribution and the real one (which concentrates all the mass at the ground truth label). Formally, the cross entropy loss computes

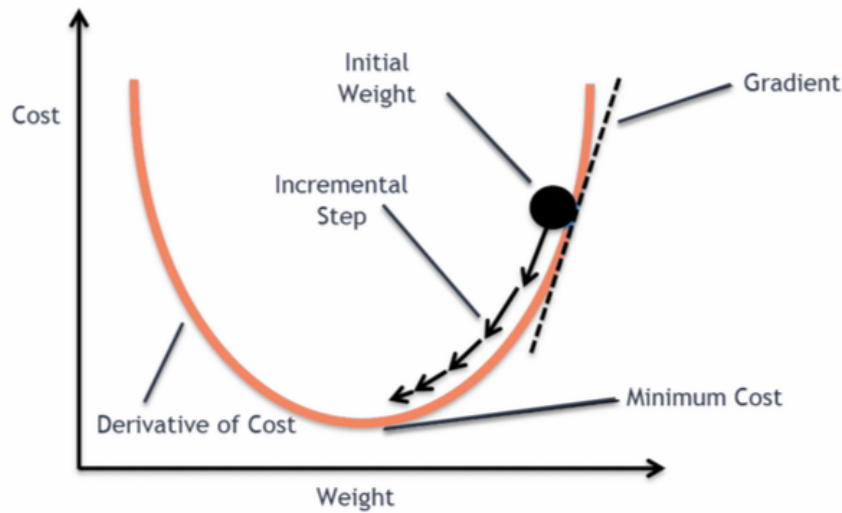


Figure 27: Illustration of the gradient descent optimization. *Figure from [clairvoyantsoft](#).*

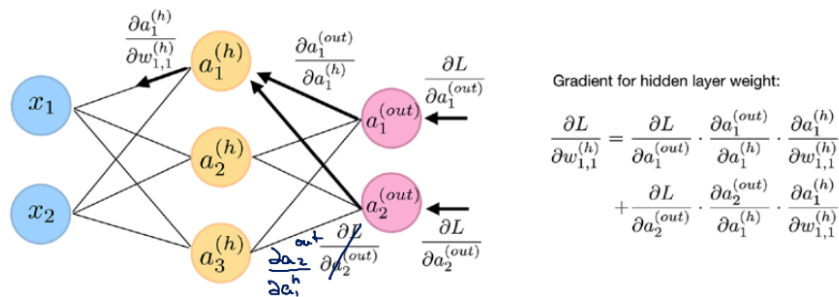


Figure 28: Backpropagation illustration. *Figure from [Raschka et al. \[2022\]](#)*

$$L(\mathcal{X}, f) = \frac{1}{|\mathcal{X}|} \sum_{(x,y) \in \mathcal{X}} \sum_{c \in C} \delta_c(y) \log(f(x))$$

with $\delta_c(y)$ equal 1 if and only if $c = y$, otherwise its value is 0.

Equation (22) is often minimized via gradient descent or variants of it.⁶ Briefly, gradient descent updates the weight parameters, w , of the network by taking steps in the direction of highest descent of the loss, which coincides with the negative direction of the gradient of L with respect to w . Figure 27 illustrates the main idea behind gradient descent. Formally, gradient descent updates the weights as follows

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w_t}. \quad (23)$$

The parameter η is called the learning rate and it determines the scale of the steps. Thus, in order to optimize the neural network we need to be able to compute the gradient of L with respect to the parameters w . This is done in the backpropagation step of the training, which exploits the chain rule. Figure 28 illustrates the backpropagation step.

⁶The blog [medium.com](#) summarizes some of the most popular optimizers

```
for epoch in range(number_epochs):
    for data in dataloader:
        # get batch of inputs and labels
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        #forward pass
        outputs = net(inputs)

        #Compute loss value
        loss_value = loss_function(outputs, labels)

        # Perform backpropagation step
        loss_value.backward()

        #Update the parameters
        optimizer.step()
```

Figure 29: Standard neural network training loop implemented in pytorch.

Figure 29 shows the conventional training loop of a neural network implemented in pytorch.

The specific meaning of the commands can be found [here](#). Now we will describe the training loop: the loop iterates over a fixed number of epochs⁷ (line 0) and inside each epoch iterates over the training data set in batches (line 1). During each iteration, the network computes the outputs for the current batch of data (line 9), followed by the calculation of the loss (line 12). Subsequently, the gradients of the loss with respect to the network's parameters are computed via backpropagation (line 15). Finally, the optimizer updates the parameters of the network using the computed gradients (line 18).

4.2 Neural Networks

In this section we will describe the structure of a neural network and some of their basic building blocks.

4.2.1 Multilayer Perceptron

The building block of the multilayer perceptrons (MLP) consists of the composition of a linear mapping followed by a non-linear function. Formally, given $x \in \mathbb{R}^n$, the building block is expressed as

$$f(x) = \sigma(Wx + b) = \sigma\left(\sum_{i=1}^{n_{out}} \langle w_i, x \rangle + b_i\right), \quad (24)$$

where $W \in \mathbb{R}^{n_{out} \times n_{in}}$ and $b \in \mathbb{R}^{n_{out}}$ are the so-called weight and bias matrices respectively. The values $n_{in}(=n)$ and n_{out} represent the input and output dimension (also called width)

⁷An epoch in machine learning refers to an iteration where the complete training data set has been pass through the algorithm.

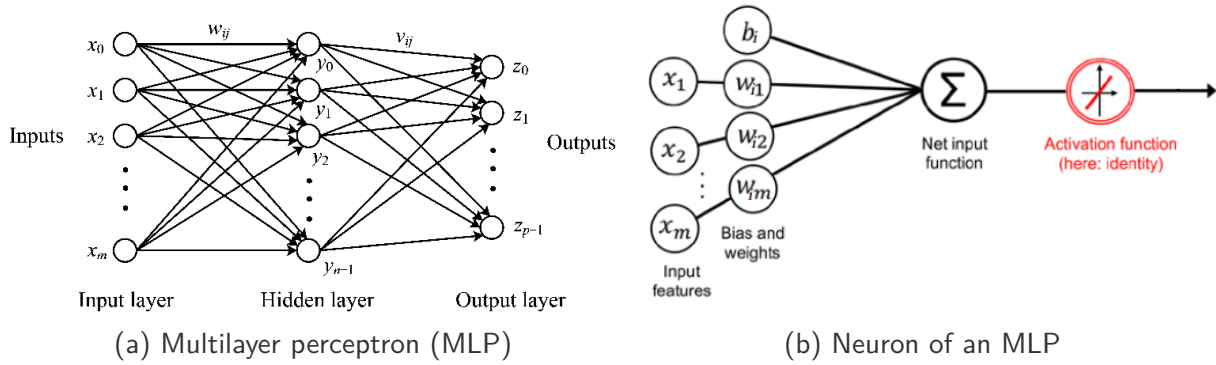


Figure 30: 30a Visualization of an MLP. Each circle represents a neuron of an MLP. 30b Processing at a single neuron of an MLP. Here $\{w_{ij}\}_{j=1}^m$ are the entries of a single row of the weight matrix W , and b_i is the i th entry of the bias. Figure 30a from cloudfront.net and 30b modified from [Raschka et al. \[2022\]](#)

respectively. The entries of W and b , often referred as weights of the neural network, are the parameters that the network learns in order to achieve the final task.

The function σ , also known as activation function,⁸ is a non-linear function which may also be parameterized with learnable parameters. In Section 4.2.3, we list some of the most popular activation functions. The activation functions are crucial, since provide extra flexibility to learn the task. If there were not any non-linearities we would just learn a linear function.

We can interpret f in (24) as a template matching function. Consider each row w_i of the weight matrix W as a template pattern. The inner product between w_i and x measures the similarity or alignment between the input vector x and the template w_i . If the inner product is high, it suggests that the input x aligns well with the template w_i , indicating a strong match. The non-linearity determines the strength of the matching, effectively deciding whether the neuron is activated or not. In essence, each row of the weight matrix represents a template pattern, and the function f processes how well the input x matches with these patterns

Function (24) constitutes a single block of the Multilayer Perceptron (MLP), and the MLP itself is a composition of functions of this form:

$$\text{MLP} = f_L \circ \dots \circ f_1.$$

Each function f_i represents a layer of the MLP. An MLP must at least have three layers: input layer (which contains the input), a hidden layer and the output layer.

The layer structure of the neural network facilitates the computation of derivatives using the chain rule. For two differentiable functions, f and g , the chain rule states that

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(y)}{\partial y} \frac{\partial g(x)}{\partial x}.$$

Thus, to calculate the derivative of the MLP function with respect to some weight, w_ℓ , at layer ℓ , we can apply the chain rule recursively. This allows us to compute $\frac{\partial \text{MLP}}{\partial w_\ell}$ efficiently by

⁸The term activation function is inspired by the biology, since the output of the neuron would determine if the neuron is active or not.

-	-	1	2	3	4	-	-	
7	6	5	-	-	-	-	-	$z_0 = x_0 w_0 = 5$
-	7	6	5	-	-	-	-	$z_1 = x_0 w_1 + x_1 w_0 = 16$
-	-	7	6	5	-	-	-	$z_2 = x_0 w_2 + x_1 w_1 + x_2 w_0 = 34$
-	-	-	7	6	5	-	-	$z_3 = x_1 w_2 + x_2 w_1 + x_3 w_0 = 52$
-	-	-	-	7	6	5	-	$z_4 = x_2 w_2 + x_3 w_1 = 45$
-	-	-	-	-	7	6	5	$z_5 = x_3 w_2 = 28$

Figure 31: Discrete 1d convolution of $x = [1, 2, 3, 4]$ with $w = [7, 6, 5]$ to yield $z = [5, 16, 34, 52, 45, 28]$. We see that this operation consists of “dragging” w over x , multiplying elementwise, and adding up the results (dragged inner product). *Figure from [Murphy \[2022\]](#).*

backpropagating the derivatives through the layers in reverse order, requiring only the outputs and derivatives at layers $L \leq l < \ell$. Figure 28 illustrates the backpropagation.

Despite the apparent simplicity of the family of MLP functions, they possess the remarkable ability to approximate a wide variety of functions with arbitrary precision, given a sufficient number of neurons or layers. This property is known as the “universal approximation theorems”⁹.

4.2.2 Convolutional Neural Networks (CNN)

In this section, based on chapter 14 of [Murphy \[2022\]](#), we will discuss a popular subclass of the MLP architecture used to process data which has spatial structure, like images. In contrast to MLPs, which learn functions mapping “unstructured” input vectors to outputs, the convolutional neural networks (CNNs) exploit the structure of the data to define inductive biases which are encoded in the weight matrix.

The CNNs architecture is equal to the MLP one but replaces the matrix multiplication with a convolution operation. First we will define the convolution operation for the 1-dimensional case. Let $w \in \mathbb{R}^d$ and $x \in \mathbb{R}^n$. We define the convolution¹⁰ between two vectors as

$$[w \star x]_i = \sum_{u=1}^d w_u x_{i+u}. \quad (25)$$

For the moment we will ignore the boundary effects. We can interpret this operation as an inner product where the indices of w have been “dragged” over x as illustrated in Figures 31 and 32.

Analogously, we can define the 2d-convolution as (see 33 for an illustration)

$$[W \star X]_{ij} = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{uv} x_{i+u, j+v}. \quad (26)$$

⁹See Wikipedia: https://en.wikipedia.org/wiki/Universal_approximation_theorem.

¹⁰In the deep learning literature, the term “convolution” is usually used to mean cross correlation. The convolution operation reverses the indices of the w vector, i.e. the convolution is technically defined as $[w \star x]_i = \sum_{u=1}^d w_{d-u} x_{i+u}$. Nonetheless, we will use the term convolution for cross correlation as it is done in the deep learning literature.

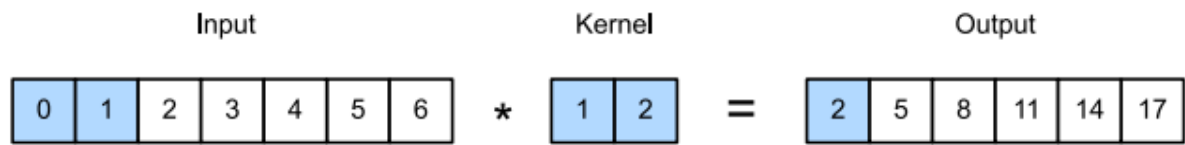


Figure 32: Illustration of 1d cross correlation, here referred as 1d convolution. *Figure from Murphy [2022].*

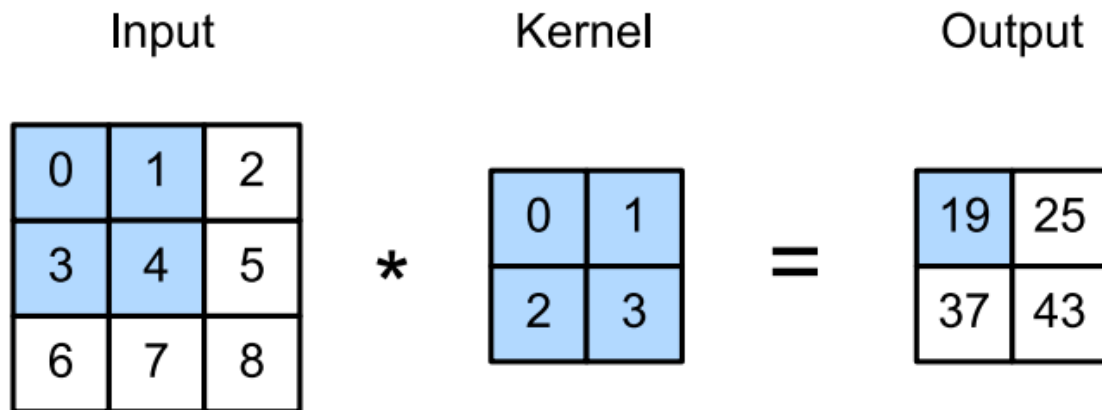


Figure 33: Illustration of 2d cross correlation, here referred as 2d convolution. *Figure from Murphy [2022].*

As it happened for the MLP, convolutional neural networks (CNNs) leverage the concept of convolution as a form of template matching, where the template pattern corresponds to a small patch of the input data. In this context, the convolution operation measures the similarity between each patch of the input data x and the filter W . Indeed, entry i, j of the convolution output will be high if the corresponding patch of x centered on (i, j) is similar to W . This is not a coincidence since as we have mentioned at the beginning, CNNs are a subset of MLPs. Indeed, the convolution with a filter W can be expressed as matrix multiplication using Toeplitz-like matrices. We will not delve into the details of this mathematical equivalence here, Figure 34 illustrates an example of expressing convolution as matrix multiplication using a Toeplitz-like matrix.

In summary, we observe that the CNNs are like MLPs which have a special matrix structure with weight sharing across the spatial locations. The weight sharing reduces the computational costs and moreover implements translation equivariance. Translation equivariance implies that a shift on the entries of the feature vector, will shift the output convolution in the same way. This is useful for tasks such as image classification, where the goal is to classify if an object is present, regardless of its location (see Figure 35).

Now we will talk about the boundary effects which have been ignored till now. As we have seen, the convolution operation slides the filter vector w over the feature vector x , which requires that spatially w fits inside of x at each location. Nonetheless, at the borders that may not happen. In general, convolving a $f_h \times f_w$ filter over an image of size $x_h \times x_w$ produces an output of size $(x_h - f_h + 1) \times (x_w - f_w + 1)$, as exemplified by Figure 27.

To mitigate this issue and ensure that the output has the same size as the input, we can employ padding techniques. One common approach is zero-padding, where we add a border

$$\begin{aligned}
 Y &= \begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \star \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix} \\
 &= \begin{pmatrix} w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5 & w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6 \\ w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8 & w_1x_5 + w_2x_7 + w_3x_8 + w_4x_9 \end{pmatrix}
 \end{aligned} \tag{27}$$

$$y = Cx = \left(\begin{array}{ccc|ccc|ccc} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{array} \right) \star \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix} \tag{28}$$

$$= \begin{pmatrix} w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5 \\ w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6 \\ w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8 \\ w_1x_5 + w_2x_7 + w_3x_8 + w_4x_9 \end{pmatrix}$$

Figure 34: Convolution in (27) can be expressed as a matrix multiplication using a Toeplitz-like matrix (28). We can recover the 2×2 output by reshaping the 4×1 vector y back to Y . Example from [Murphy \[2022\]](#).

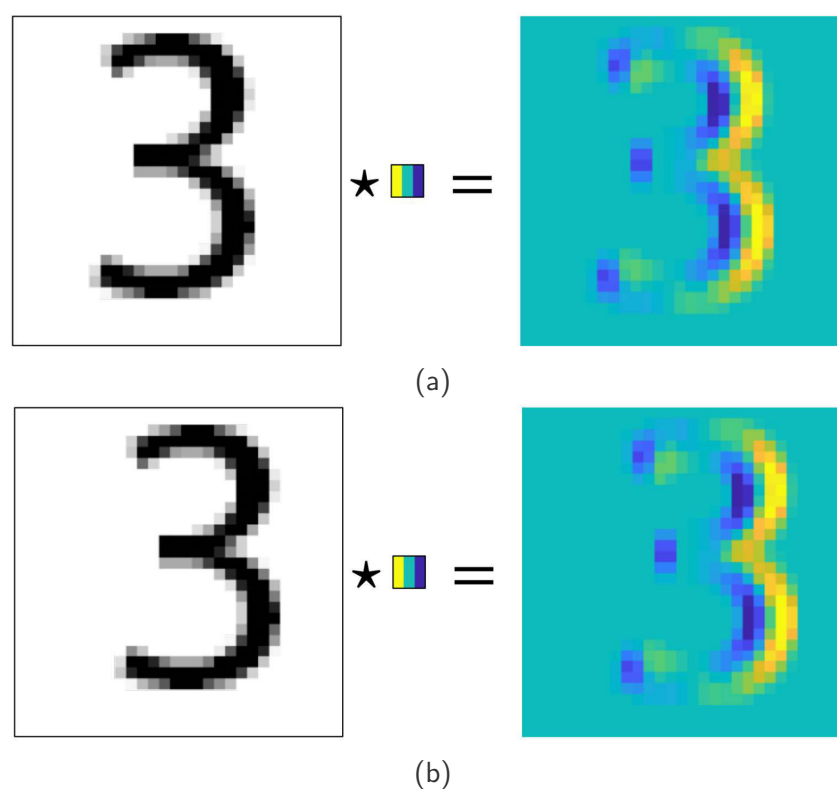


Figure 35: The convolution operation is translation equivariant. Indeed, translating the 3 and applying the filter is equivalent to first convolving the 3 with the filter and later applying the translation to the response. *Figure from doc.ic.ac.uk.*

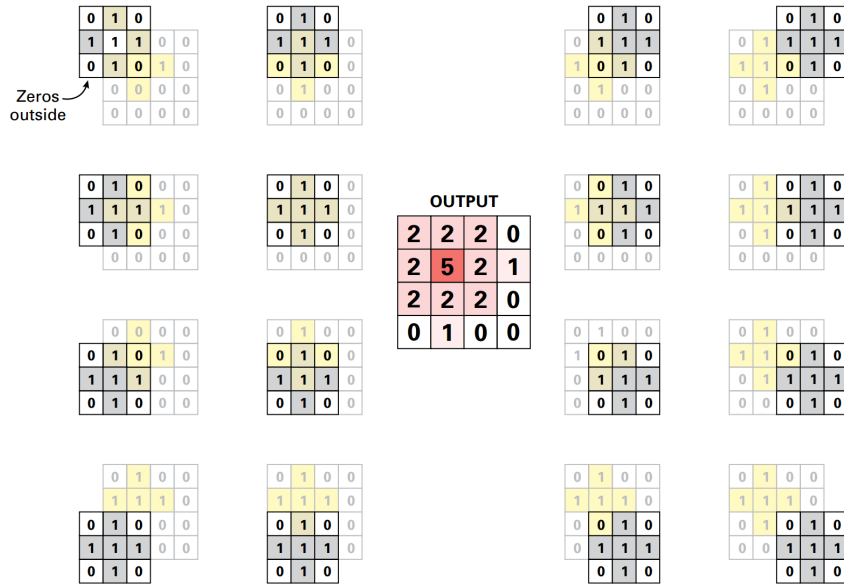


Figure 36: The use of zero-padding ensures that the output is the same size as the input. *Figure from [Murphy \[2022\]](#).*

of zeros around the input image before performing the convolution operation, as depicted in Figure 36. This effectively increases the size of the input image, allowing the filter to slide over the entire input and produce an output of the desired size.

Indeed, in addition to standard convolutions where the filter slides over the input with steps of one unit, we can also perform strided convolutions where we skip every s -th input entry (see Figure 37). Strided convolution is a useful technique for reducing the spatial dimensions of the output while maintaining information content, effectively downsampling the input feature maps.

In general, for an input of size $x_h \times x_w$ convolved with a filter of size $f_h \times f_w$, with padding on each side of size p_h and p_w and strides of size s_h and s_w , will produce an output of size

$$\left\lfloor \frac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{x_w + 2p_w - f_w + s_w}{s_w} \right\rfloor.$$

For instance for the example shown in Figure 37 the size is

$$\left\lfloor \frac{5 + 2 - 3 + 2}{2} \right\rfloor \times \left\lfloor \frac{7 + 2 - 3 + 2}{2} \right\rfloor = \left\lfloor \frac{6}{2} \right\rfloor \times \left\lfloor \frac{8}{2} \right\rfloor = 3 \times 4.$$

Finally, we define the convolution where the input has multiple channels, C . For instance, RGB-images are composed by 3 channels, one for each color. In this case, the convolution is defined as

$$[W \star X]_{ij} = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} w_{uvc} x_{i+u, j+v, c}. \quad (29)$$

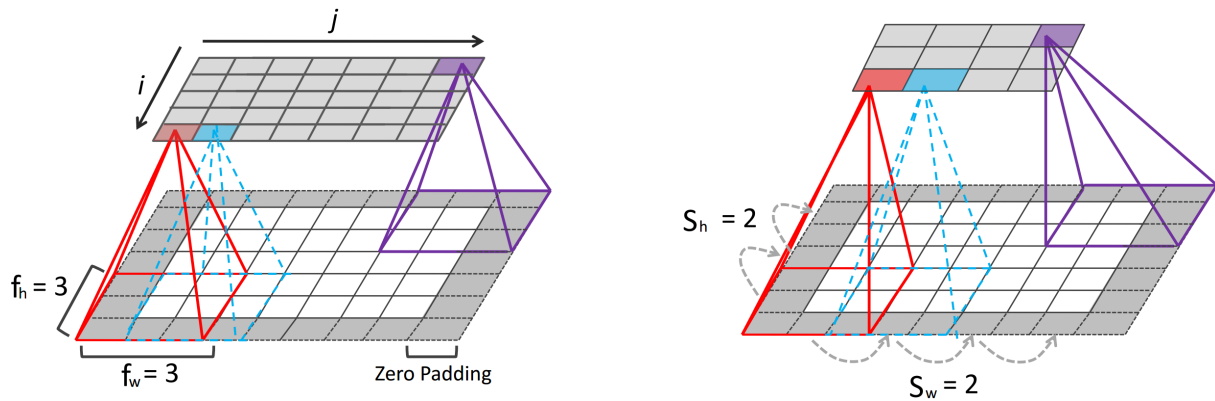


Figure 37: Illustration of padding and strides in 2d convolution. (a) Applying zero padding on a 5×7 input convolved with a 3×3 filter to create a 5×7 output. (b) Now we use a stride of 2, so the output has size 3×4 . *Figure from [Murphy \[2022\]](#)*

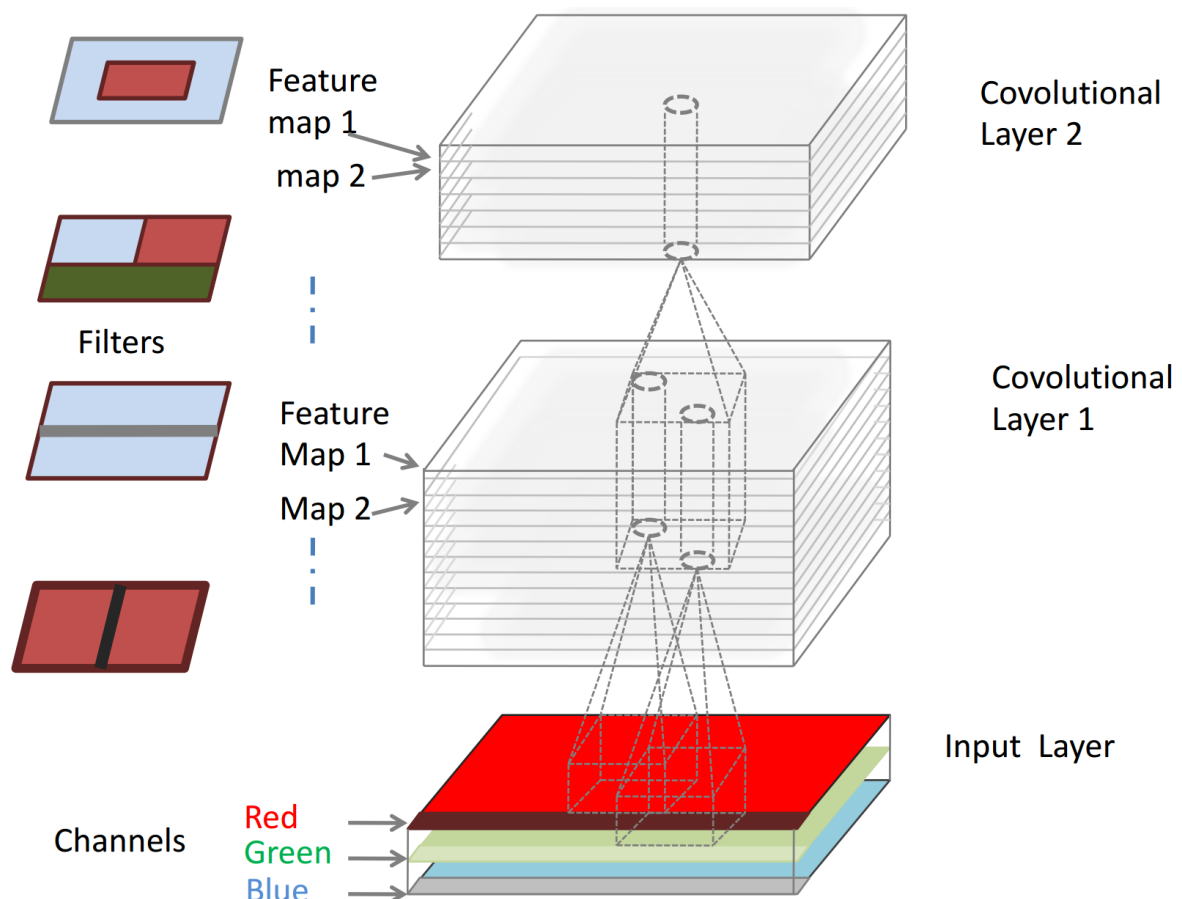


Figure 38: Illustration of a CNN with 2 convolutional layers. The input has 3 color channels. The feature maps at internal layers have multiple channels. The cylinders correspond to hypercolumns, which are feature vectors at a certain location. *Figure from [Murphy \[2022\]](#)*.

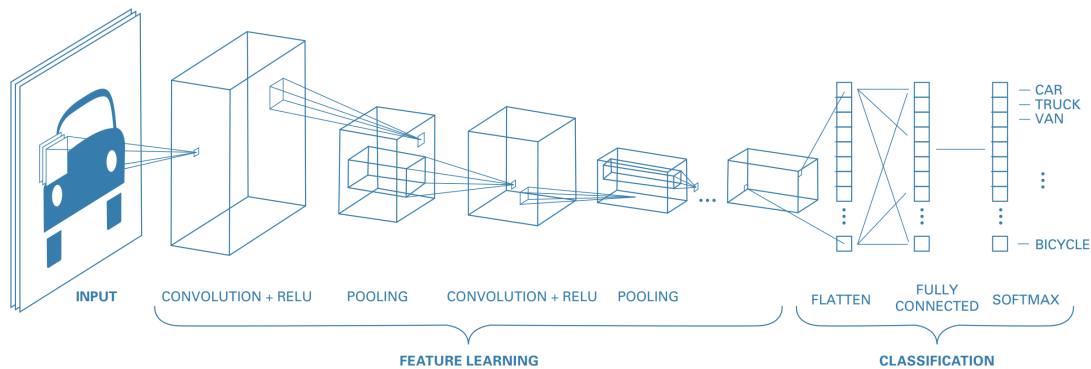


Figure 39: CNN illustration for classification. *Figure from Murphy [2022].*

4.2.3 Activation functions

In this section we provide a list of the most popular activation functions. A longer list can be found [here](#). All the functions listed are applied pointwise on each entry, except the softmax function.

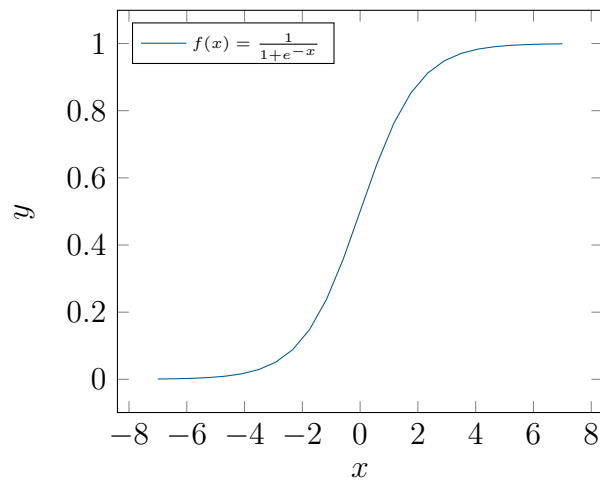
Sigmoid function: $f(x) = \frac{1}{1+e^{-x}}$. See Figure 40a. Typically used when the desired output is between 0 and 1. For inputs with absolute value higher than 3, the gradient is close to 0. As the gradient value approaches zero, the network ceases to learn and suffers from the vanishing gradient problem.

Softmax function: $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ such that $f(x)_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{-x_j}}$. Similar to the sigmoid, it is usually used to model probabilities, since it maps each entry of the output vector to the range $(0, 1)$ and the sum of the entries is equal to one. That is, $\sum_{i=1}^n f(x)_i = 1$ and $f(x)_i \in (0, 1)$.

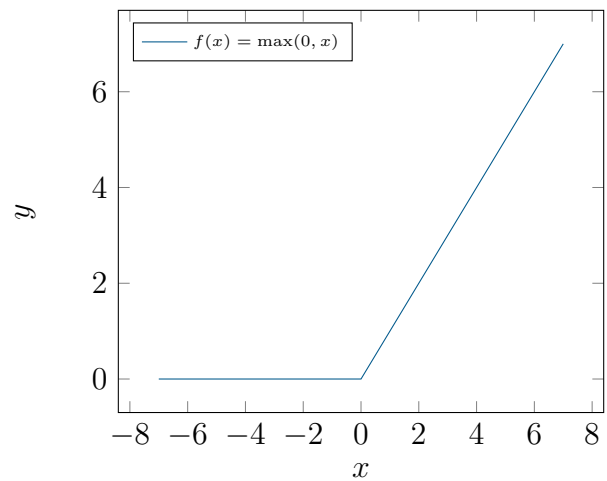
Rectified Linear Unit (ReLU) function: $f(x) = \max(0, x)$. See Figure 40b. In this case the gradient does not saturate, but is constant (equal to 1) for positive values. Moreover, it is computationally efficient to compute. However, a limitation of the ReLU function is its tendency to "die" for negative input values. When the input is negative, the output of the ReLU function becomes zero, effectively killing the neuron and causing it to cease learning. This issue may reduce the model's ability to fit the data, particularly if a large portion of the input data is negative.

LeakyReLU function: $f(x) = \begin{cases} ax & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$ (usually $a = 0.01$). In some cases a is learned. See Figure 40c. Leaky ReLU is an improved version of ReLU, which solves its problems by having an small positive slope on the negative region.

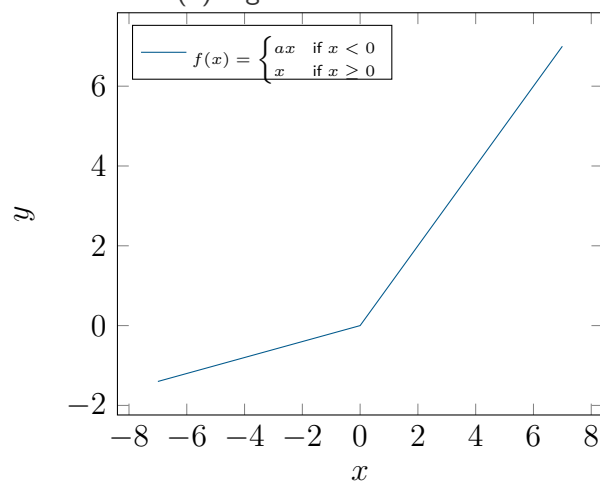
ELU function: $f(x) = \begin{cases} \alpha(e^x - 1) & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$ where α is a learnable parameter. See Figure 40d. It is an smooth alternative to the ReLU function though it requires a higher computational cost. Moreover, for higher values of x , the gradient exploits.



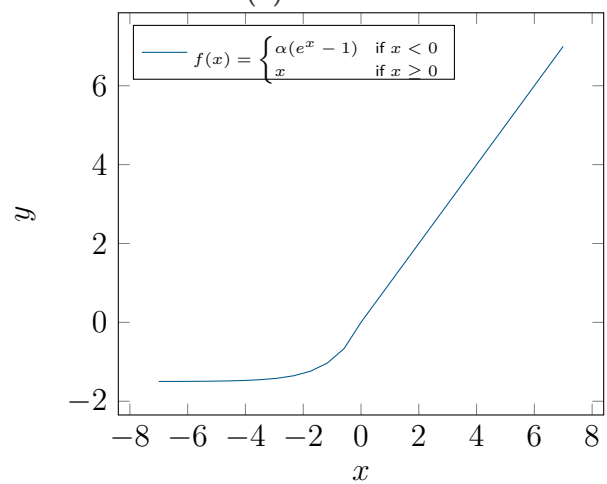
(a) Sigmoid function.



(b) ReLU.



(c) Leaky ReLU.



(d) ELU.

Neural Network Activation Functions

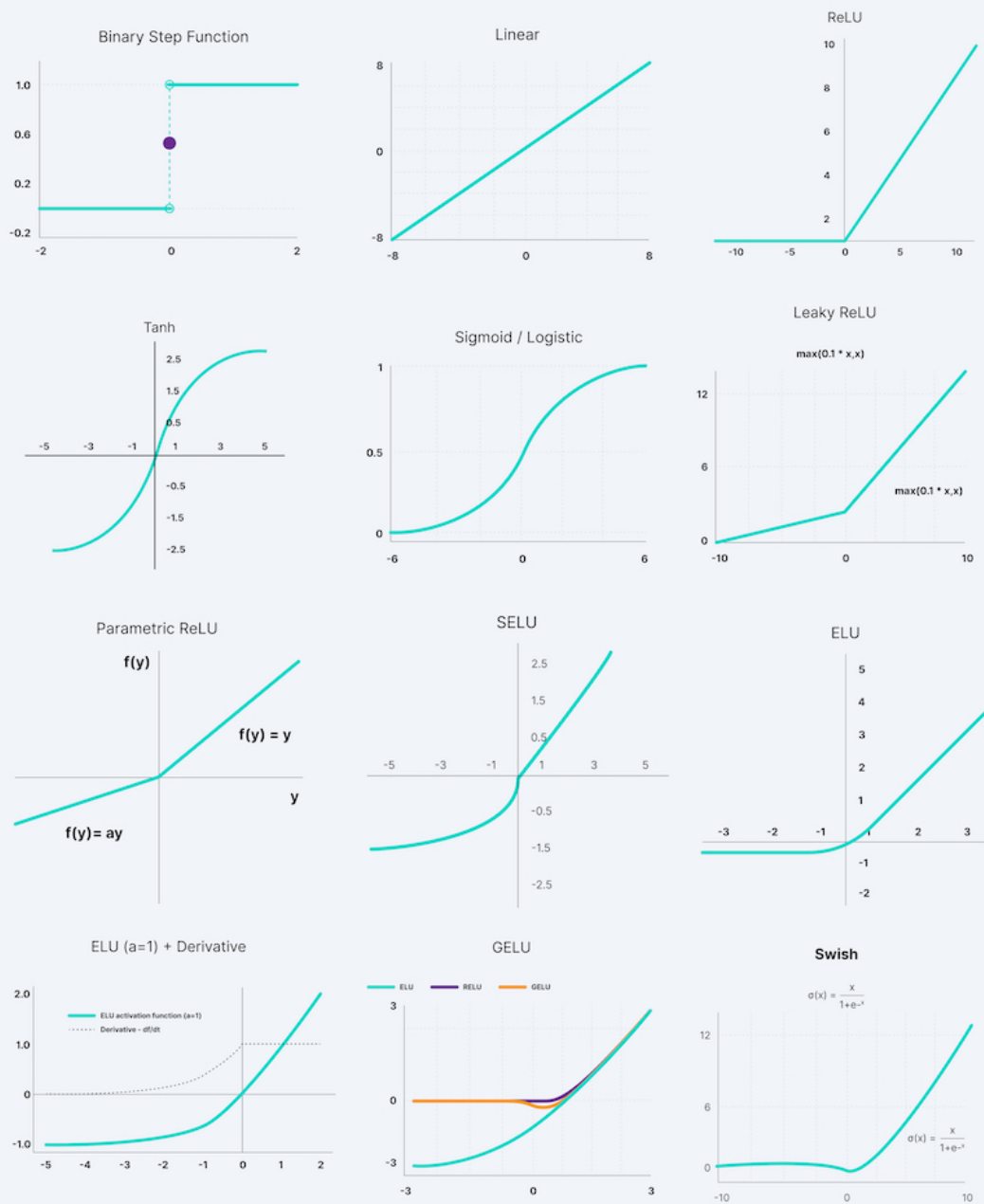


Figure 41: Neural network activation functions. *Figure from v7labs.com.*

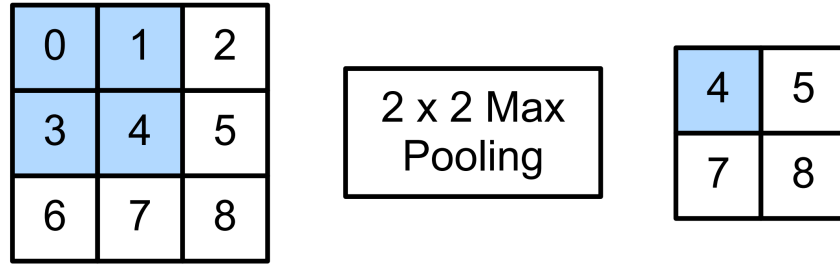


Figure 42: Illustration of max-pooling with a 2×2 filter and a stride of 1. *Figure from [Murphy \[2022\]](#).*

4.2.4 Other layers

In this section we cover two other popular layers:

Pooling layers These layers are commonly used in image processing. As discussed earlier, convolutions provide translation equivariance, effectively preserving information about the location of input features. However, in certain scenarios, we seek invariance to location, meaning the output should not depend on the precise location of an object within the image. Pooling layers address this need. For instance, the max-pooling layer calculates the maximum value from its input values (refer to Figure 42). Another option is average pooling, which computes the average value. See Figure 42 for an illustration of the max-pooling process.

Batch norm layer The batch norm layer performs a parameterized standardization of the data. Given a sample z , the batch norm layer computes

$$\tilde{z} = \gamma \odot \hat{z} + \beta,$$

where γ and β are two learnable parameters, and $\odot$ denotes pointwise multiplication. The standardized activation vector $\hat{z}$ is obtained using the following formulas:

$$\begin{aligned}\hat{z} &= \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \\ \mu_B &= \frac{1}{|B|} \sum_{y \in B} y \\ \sigma_B^2 &= \frac{1}{|B|} \sum_{y \in B} (y - \mu_B)^2\end{aligned}$$

where B is the minibatch containing sample z , μ_B is the mean of the activations for this batch, and σ_B^2 is the corresponding variance. Thus, $\tilde{z}$ is the standardized activation vector of z , i.e., $\hat{z}$, which has been shifted and scaled by γ and β .

During training, batch statistics are readily available. However, at testing time, when only a single input is available, computing meaningful batch statistics is not feasible. Hence, during testing, the batch statistics of the training set are utilized. Although the exact reasons for the benefits of batch normalization during training are not fully understood, it can improve training performance.

References

- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Information science and statistics. Springer, New York.
- Damrich, S. and Hamprecht, F. A. (2021). On umap's true loss function. *Advances in Neural Information Processing Systems*, 34:5798–5809.
- Gãlron, A. (2019). *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly UK Ltd., Beijing China ; Sebastopol, CA, 2nd ed edition edition.
- Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137. Conference Name: IEEE Transactions on Information Theory.
- Maaten, L. v. d. and Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605.
- Marsland, S. (2014). *Machine Learning: An Algorithmic Perspective*. Chapman and Hall/CRC, 2 edition.
- McInnes, L., Healy, J., and Melville, J. (2018). Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Murphy, K. P. (2022). *Probabilistic machine learning: an introduction*. Adaptive computation and machine learning series. The MIT Press, Cambridge, Massachusetts.
- Mãijller, A. and Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media, Incorporated.
- Raschka, S., Liu, Y. H., Mirjalili, V., and Dzhulgakov, D. (2022). *Machine Learning with Pytorch and Scikit-Learn Develop Machine Learning and Deep Learning Models with Python*. Packt Publishing, Limited, Birmingham. OCLC: 1295278229.